TABLE OF CONTENT

SL NO:	CONTENT	Page No:
CHAPTER - I	**INTRODUCTION**	1
CHAPTER - II	**SYSTEM ANALYSIS**	5
CHAPTER – III	**REQUIREMENTS SPECIFICATION**	8
CHAPTER - IV	**TECHNOLOGY STACK**	10
CHAPTER - V	**SYSTEM DESIGN**	17
CHAPTER - VI	**DETAILED IMPLEMENTATION**	30
CHAPTER - VII	**PROJECT MODULES**	36
CHAPTER - VIII	**SYSTEM TESTING**	40
CHAPTER - IX	**DEPLOYMENT AND MAINTENANCE**	46
CHAPTER - X	**CONCLUSION & FUTURE SCOPE**	51
CHAPTER - XI	**SOURCE CODE**	55
	REFERENCES	84
	APPENDIX	85

LIST OF TABLES

Table No.	Title	Page
3.1	Hardware Requirements	9
3.2	Software Requirements	10
3.3	User Roles and Permissions Matrix	11
5.1	Database Table: Users	21
5.2	Database Table: Events	22
5.3	Database Table: Registrations	23
7.1	Module Overview	37
8.1	Authentication Test Cases	41
8.2	Event Management Test Cases	42
8.3	Registration Test Cases	43
8.4	Search and Recommendation Test Cases	44
8.5	Admin Panel Test Cases	44

LIST OF FIGURES

Figure Number	PARTICULARS	PAGE NUMBERS
5.1	Three-Tier System Architecture Diagram	18
5.2	Frontend Component Hierarchy	19
5.3	Backend Route and Controller Architecture	20
5.4	Entity-Relationship Diagram	21
5.5	DFD Level 0 – Context Diagram	25
5.6	DFD Level 1 – Process Decomposition	25
5.7	Use Case Diagram	26
5.8	Sequence Diagram – User Registration	27
5.9	Sequence Diagram – Event Creation and Approval	28
5.10	JWT Authentication Flow	29
7.1	Module Interaction Diagram	38

CHAPTER 1:

INTRODUCTION

1.1 Project Background

Campus life at any college or university is really busy with a lot of events happening all the time. Things like guest lectures sports games between colleges, cultural festivals, coding competitions, job placement events and seminars organized by teachers. These events are not just for fun they also help students make connections improve their communication skills and learn about things outside of their regular classes.. In most colleges information about events is shared in a pretty disorganized way. Through notices on bulletin boards emails from departments, announcements in class and messages on WhatsApp groups.

This way of sharing information can cause problems. For example a third-year engineering student who is interested in a data science workshop might not even hear about it if the announcement is only posted on the statistics departments bulletin board. A teacher who is in charge of approving events might get requests by email in person and on paper. There is no simple way to keep track of what has been approved and what is still pending.. After a big event like a hackathon with 300 people the organizer has no easy way to collect feedback from attendees or look at trends to plan the next event.

That is why we created the College Event Aggregator. To replace this system with a simple web-based platform that works for everyone involved in campus events. The basic idea is easy to understand: we want to put all event-related activities. Creating events finding events signing up for events approving events sending out notifications and looking at statistics. Into one application that can be accessed from any browser on any device. What makes this project special is that it includes features. Like personalized event suggestions, easy search, automatic category sorting and important notifications. That actually help users find events they are interested, in instead of just waiting for them to search.

The platform is built in a way that makes it easy to manage with three parts: a user interface built with React a back-end built with Node.js that handles the logic and database and a special service built with Python that handles the smart features. We use PostgreSQL, hosted on the Supabase platform to store all the data. This way each part can be developed, tested and scaled independently which makes it easier to maintain and improve the platform.

1.2 Motivation

Several pain points I faced during college motivated this project:

- Information Asymmetry. Students in one department often do not know about events in departments. This leads to missed opportunities for learning experiences. The platform solves this by collecting all events into a directory.
- Manual Approval Bottlenecks. In some institutions events need approval from faculty or administrators before they can be shared. The approval process is usually done on paper or via email. This can cause delays. Lost files. A digital approval system with status updates fixes these issues.
- Low Engagement Due to Noise. When students get many irrelevant event notifications they stop paying attention to all notifications. A recommendation engine that learns what each student likes and a system that shows relevant alerts can help.
- Absence of Post-Event Analytics. Currently organizers and administrators do not have a way to measure event success. They do not know how many people registered or attended or what the feedback ratings were. The platforms dashboard and analytics provide this information.
- Academic and Technical Interest. This project was a chance to combine technologies like React, Express and FastAPI into a single system. Building a recommendation engine, search and access

control system helped me learn beyond the classroom.

I worked on integrating React with TypeScript Express with Prisma and FastAPI, with scikit-learn.

Building a recommendation engine and designing a role-based access control system were part of this project.

1.3 Problem Definition

The problem is this: how can a college bring together all the parts of campus events. From when someone first suggests an event to when it happens and afterwards. In a way that everyone involved can use that shows people the events that matter to them and that helps the people in charge keep track of everything.

The old way of doing things does not work for three reasons:

1. It is hard for students to find events that are not connected to their friends or the department they are in.

2. The people who organize events and the administrators do not have a system to create approve and keep track of events.

3. There is no way to look at data to see how many people go to events how engaged students are or what kinds of events are popular over time.

The College Event Aggregator solves these problems by doing a things: it helps users find events, with search and filters and suggestions it helps organizers and administrators with automatic approval processes and reminders and it helps with analytics by showing real time statistics on dashboards, which is part of the College Event Aggregator the College Event Aggregator has all these features.

1.4 Objectives

The primary objectives of this project are as follows:

- **Design and develop a responsive web platform** that allows students to browse, search, filter, and register for college events from any device.
- **Implement a role-based access control system** with four distinct roles – Student, Organizer, Faculty, and Admin – each with a clearly defined set of permissions governing what actions they can perform.
- **Build an event approval workflow** where events created by organizers are submitted to faculty or administrators for review before being published to the general student body.
- **Develop a hybrid recommendation engine** that combines content-based filtering (based on user category preferences), popularity scoring (based on registration activity), and time-aware suggestions (prioritizing imminent events) to deliver personalized event recommendations.
- **Implement NLP-powered search** with query preprocessing, stopword removal, stemming, synonym expansion, and intent detection, enabling students to find events using natural language queries.
- **Create role-specific dashboards** that provide students with upcoming events and recommendations, organizers with registration and attendance analytics, and administrators with system-wide statistics and user management tools.
- **Ensure security** through industry-standard practices including JWT authentication with token rotation, bcrypt password hashing, rate limiting, Helmet security headers, and input validation.
- **Deploy the platform** using Supabase for database hosting, ensuring the system is accessible beyond the local development environment.

1.5 Scope of the Project

The scope of this project encompasses the following functional areas:

- **User Management**: Registration, login, profile management, and role assignment.
- **Event Lifecycle**: Creation, editing, approval/rejection, publishing, and deletion of events.
- **Event Discovery**: Browsing events with filters for category, type, date range, and status; full-text search with NLP augmentation.
- **Registration Management**: Students registering for events, organizers managing attendee lists, waitlisting when capacity is full.
- **Notification System**: System-generated alerts for registration confirmations, event reminders, approval status updates, and trending event highlights.
- **Dashboards**: Role-specific analytics views with real-time statistics.
- **Admin Panel**: User management (role changes, account suspension), system configuration, and event oversight.

The following areas are explicitly **outside** the current scope but are identified as future enhancements: payment integration for paid events, real-time chat between organizers and attendees, calendar synchronization with Google Calendar or Outlook, and QR-code-based attendance tracking.

1.6Applications

The platform, while developed in the context of a single college, is designed with reusability in mind. Its potential applications include:

- **Universities and Colleges**: The primary use case. Any educational institution with multiple departments and a regular schedule of events can deploy this platform to centralize event communication.
- **Corporate Organizations**: Companies that organize internal events – training sessions, team outings, town halls – can adapt the platform's event lifecycle and registration modules.
- **Clubs and Societies**: Student organizations, hobby clubs, and professional societies can use the platform to manage their event calendars and member registrations.
- **Community Centers**: Local community organizations hosting workshops, classes, and social gatherings can benefit from the registration and notification features.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing System

Before starting the development of the proposed system, it was necessary to understand how event management usually works in colleges. In many institutions there is not a single platform where all events are listed together. Instead, information about events is normally shared in different ways and in different places.

One common method is the use of **physical notice boards**. When an event is organized, a poster or printed notice is usually pinned on the department notice board. This method has been used for a long time and still works in many colleges. However it also has some problems. Students from other departments may not visit that notice board regularly, so they may never even see the announcement.

Another common way is through **WhatsApp groups or social media pages**. Many clubs and student groups share event details there. But the issue with these groups is that there are a lot of messages everyday. Because of that, sometimes important event messages get lost between other chats. Students might scroll past them or not notice them properly.

Sometimes organizers also share **Google Forms links** for registration. These forms help in collecting names and details of participants. But the problem is that the form link is usually shared separately from the announcement. If someone misses the message or the link, they cannot easily find it again.

Some colleges do have their own **internal portals or websites** where events are posted. But many of these portals are quite basic and sometimes not updated regularly. In some cases the interface is also not very user friendly, so students prefer to rely on WhatsApp or word of mouth instead.

So overall, the current system for managing college events is somewhat scattered. Information exists, but it is spread across different places and platforms.

2.2 Drawbacks of Existing System

From observing the current system, a few problems become quite clear.

One major issue is the **lack of centralization**. Event information is spread across notice boards, emails, WhatsApp groups and sometimes social media pages. Because of this students often miss events simply because they didn't see the announcement.

Another problem is related to **registration management**. When organizers use Google Forms or other external tools, they collect responses but there is usually no proper system to manage that data later. For example checking attendance or participation numbers can become a bit difficult.

The **approval process** for events is also mostly manual in many colleges. Organizers usually have to contact faculty members through emails or sometimes paper forms. This process works, but it can be slow and sometimes confusing, especially when multiple approvals are needed.

There is also **no personalization** in the current system. Every student receives the same information regardless of their interests. For example a student interested mainly in coding competitions may still receive announcements about sports or cultural events.

Another limitation is the **lack of analytics or feedback data**. After the event is completed, organizers may not have clear information about attendance numbers or how students felt about the event. Without such information it becomes harder to improve future events.

Finally, many existing systems are **not very mobile friendly**. Since most students use their phones to check information, this can become a small but noticeable problem.

2.3 Proposed System

To address the issues mentioned above, the **College Event Aggregator** system is proposed in this project.

The idea is to create a single web platform where all college events can be displayed and managed in a more organized way. Instead of checking multiple sources, students would be able to visit one platform and see the list of upcoming events.

Through this system students can browse events, search for specific types of activities, and register directly for the events they are interested in. This makes the process a bit easier compared to the existing methods.

Event organizers will also be able to create events through the platform. These events can then be reviewed and approved by faculty members or administrators before they become visible to students.

The system can also include some additional features like event filtering and recommendation. For example the platform may suggest events to users based on their interests or previous activity. This can help students discover events they might not have noticed otherwise.

Another feature is the **notification system**, which can inform users about event approvals, reminders, or registration updates.

Overall, the proposed system tries to make event information more organized and easier to access for everyone in the college.

2.4 Advantages of Proposed System

The proposed system offers several improvements compared to the existing approach.

First, it provides a **centralized platform** where students can see all events in one place. This means they do not have to check different sources like notice boards or WhatsApp groups again and again.

Second, the system introduces a **more structured process** for event creation and approval. Organizers can submit event details through the platform, and faculty members can review them before publication.

Another advantage is **better event discovery**. Students can search events using keywords or apply filters to find events related to their interests.

The platform can also provide **basic statistics and participation information**, which may help organizers understand how successful an event was.

Since the platform is designed as a web application, it can also be accessed easily through mobile devices as well as desktop computers.

Overall, the proposed system aims to make event management simpler and more organized for both students and organizers.

CHAPTER 3

REQUIREMENTS SPECIFICATION

3.1 Hardware Requirements

Hardware specifications In order to properly develop and run the system, hardware requirements refer to useful physical components. Due to the College Event Aggregator being a web-based application that does not require very high-end hardware or any special machines. The normal computer was the platform chosen for the development of project and tested. A computer with 4 GB of RAM and a simple processor can run this application, however, developing using a slightly better configuration is easier. For the development of this software, which involved the concurrent use of the development server, database tools, and other software, we used a system configured with 8 GB RAM and Intel Core i5 Processor. In addition to the processor and RAM, some storage space is needed to install the tools, project files, and database-associated packages. Having a stable cache and strong internet connection is also important because, we will install many dependencies and packages online and services like 'supabase' also requires an internet connection.

The general hardware requirements used for this project are shown below.

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or more
Storage	5 GB free disk space	20 GB SSD
Display	1366 x 768 resolution	1920 x 1080 Full HD
Network	Broadband Internet	Stable broadband connection
Operating System	Windows 10 / macOS 10.15 / Ubuntu 20.04	Latest stable release

These specifications are quite common for most student laptops or desktop computers, so running the system should not be a major issue.

3.2 Software Requirements

Software requirements include the programming languages, frameworks, and tools used for building the system. Since this project is a full-stack web application, multiple technologies are used together for different parts of the system.

For the frontend development, **React** along with **TypeScript** was used. React helps in building interactive user interfaces, and TypeScript helps in keeping the code more organized and manageable.

The backend of the system was developed using **Node.js and Express.js**. These technologies handle the server-side logic and manage communication between the frontend and the database.

For storing and managing the data, **PostgreSQL** was used as the database system. The database is hosted using **Supabase**, which provides a managed PostgreSQL service and also offers a free tier suitable for development purposes.

Apart from this, a separate **Python service using FastAPI** was included in the project. This service supports some intelligent features such as event recommendations and search related processing.

The development work for the project was mainly done using **Visual Studio Code**, which is a commonly used code editor for web development.

The main software tools used in this project are listed below.

Category	Software	Version
Runtime	Node.js	20 LTS
Runtime	Python	3.11+
Database	PostgreSQL (Supabase)	15+
Frontend Framework	React	18.x
Frontend Language	TypeScript	5.x
Build Tool	Vite	7.x
CSS Framework	Tailwind CSS	3.x
Backend Framework	Express.js	4.x
ORM	Prisma	5.x
Python Framework	FastAPI	0.104.x
ML Library	scikit-learn	1.x
NLP Library	NLTK	3.x
IDE	Visual Studio Code	Latest

Most of these tools are open source and freely available. Because of this, they are commonly used in academic projects as well as in real-world development.

3.3 Functional Requirements

The core purpose of the system is achieved by the Functional Requirements and they describe the essential features and functionalities that the system is able to execute. These requirements describe the actions the system should support the users to perform.

Subsystems of the system should allow for user authentication. Each user should have the ability to create an account and be able to log in to their account by using an email address and password. After successfully logging in to their account, users should be able to perform activities that are dependent on their role.

Other than user authentication, the system should allow for the management of events. Faculty members and organizers have the authority to create events on the system. They should be able to create an event and specify the event title, event description, date of the event, venue of the event, and event category.

The system must also allow user registration for events. The system should allow students to register for events. The system will record the event registrations. The system should also ensure that event

registrations for the same event are not duplicated.

Also, the system should have an event notification feature that will allow users to be notified when they have successfully registered, when an event is approved, and when an event is approaching.

Additionally, the system must satisfy the requirement of having user dashboards to display relevant information to the users. For instance, students should have dashboards that display events that are coming up, whereas organizers should have dashboards that display the number of registered participants for their events.

Another requirement of the system is the ability to search and receive recommendations. Users should be able to search events using keywords, and the system may suggest events based on user interests.

Finally, the system includes an **administration module**. Administrators can manage user accounts, update roles, and monitor events across the platform.

3.4 Non-Functional Requirements

The focus of a non-functional requirement is often a description of a system's behavior rather than a list of system features.

- Performance Users should expect load times that are reasonable. Normal operation should allow for swift browsing or registration actions to events.
- Security Encryption of stored passwords and management of user sessions through authentication tokens is required.
- Usability The interface should allow a user to navigate the system without unnecessary complications.
- Reliability In normal operation the system should avoid faults and behavior that leads to crashing or freezing.
- Maintainability Organization of the code should allow for additional system features and system improvements with minimal requirement for loss.

3.5 User Roles and Permissions

The system defines different user roles, and each role has certain permissions.

Permission	**Student**	**Organizer**	**Faculty**	**Admin**
Browse Events	Yes	Yes	Yes	Yes
Register for Events	Yes	Yes	Yes	Yes
View Dashboard	Yes	Yes	Yes	Yes
Create Events	No	Yes	Yes	Yes
Approve / Reject Events	No	No	Yes	Yes

Permission	Student	Organizer	Faculty	Admin
Manage Users	No	No	No	Yes
Access Admin Dashboard	No	No	No	Yes
Delete Any Event	No	No	No	Yes

Students mainly interact with the system by browsing events and registering for them. Organizers create events and manage participant lists. Faculty members can review and approve events before they are published. Administrators have full control over the system and can manage users and events.

CHAPTER 4

TECHNOLOGICAL FRAMEWORK

4.1 React with TypeScript

For the frontend of the system, React was chosen. React is a popular JavaScript library for developing web application user interfaces. It is resourceful for developers because it promotes the design of reusable components.

In this project, React was predominantly utilized to develop the event listing page, dashboard, login and registration pages, and other components of the user interface. The component-based structure of React is beneficial for breaking the interface into smaller, manageable sections such as event cards, a navigation bar, search filters, and forms.

Additionally, React was used with TypeScript. TypeScript is JavaScript with type annotations, making it easier to produce organized code and less manageable code and providing a structure to deal with and mitigate certain categories of programming mistakes.

The combination of React and TypeScript helped streamline the development process as well as the organization of the frontend code.

4.2 Node.js and Express

The backend part of the application was developed using Node.js along with the Express.js framework.

Node.js and Express.js Framework were used to construct the application backend.

Node.js allows JavaScript to run on servers. It is a JavaScript runtime built for use outside of a browser. The technology exists so we can create programs that run on the computer (server). When you use Node.js to create your application, you can create front-end and back-end programs because you write in JavaScript all the way.

Express.js is a minimal Node.js web application framework that can be used to create APIs and handle server side-routing logic. In this project we used Express to handle authentication of users, creation of events and registrations for events, as well as all API calls to and from the platform. Basically what happens is that the back-end Node server will take requests from the front-end, ensure data is validated before hitting the database, and then retrieve data/information from the database and send it back to the front-end for display.

4.3 PostgreSQL with Prisma ORM

PostgreSQL and PrismaORM were used to store and extract data to and from the database.

PostgreSQL was used as our database system. PostgreSQL is the database system that stores all the data for the application and allows our system to interact with it. It is an open-source relational database system that has been used in many web applications. Relational databases store information in tables. Our project needed tables to store user data, events, registrations, notifications, and feedback. Prisma is what we used to abstract some of the database tasks from developers. ORM stands for Object-Relational Mapping. Instead of writing SQL queries to interact with our database, we let Prisma handle that and we interact with our database through code.
With Prisma we are also able to control our schema and handle migrations. Prisma also allowed us to have type-safe database queries. Schema and database migrations helped us with handling the database

structure better.

4.4 FastAPI (Python Service)

Aside from the primary backend server, the setup also brought in a distinct Python

service built with FastAPI.Faster than many tools out there,
FastAPI helps build web services using Python without slowing down.
Its design keeps things clear while running fast.

FastAPI handled most of the smart functions here - suggesting events, dealing with searches. The reason Py thon came into play? Loads of tools for teaching machines how to understand words already exist in that la nguage.

Running on its own, this service links to the Node.js
backend using API calls. Behind the scenes, each request travels between them like messages passed back a nd forth. Separate processes handle their roles without merging into one system. Communication happens o nly when needed, triggered by specific inputs. The flow stays clean because tasks never overlap unnecessar ily. Each piece does its job while staying independent.

4.5 Supabase

A choice fell on **Supabase** to handle the database hosting. Through its web
based control panel, access unfolds to a fully managed PostgreSQL setup.

Supabase takes care of database server setup without any extra effort. Because of this, tasks like watching p erformance or handling connections become simpler. Accessing login details happens smoothly thanks to b uilt-in tools. Features show up exactly when needed, not before.

A playground for learners, Supabase opens its doors without cost - ideal when ideas are just beginning. Earl y tests, classroom work, tiny apps - all fit right in.

4.6 Supporting Libraries and Tools

Besides the core tech listed earlier, various helper tools came into play throughout building it. Different e xtra libraries showed up along the way, quietly doing their part behind the scenes.

Few libraries handled safety checks along with login features, whereas some tackled data checking, trackin g actions, or refining how screens looked. A handful focused on protection and sign-
in steps, meanwhile others pitched in for form reviews, event records, or visual tweaks.

Examples include:

- **bcrypt** for password hashing
- **jsonwebtoken** for handling authentication tokens
- Validation of request information handled by **express-validator**
- **helmet** for adding security headers
- Page routes handled by **react-router-
dom** on the client side. Navigating between views happens through this tool. It keeps track of where users g o. Components change based on URL paths. The app knows which screen to show at each stop

- Python tools like scikit-learn plus NLTK handle search logic inside the service. These libraries power recommendations through pattern recognition. Behind the scenes, they proce ss queries using statistical methods. Machine learning models from scikit-learn adapt to user behavior over time. Text analysis tasks rely on NLTK's linguistic rules. Together, both s upport smart matching without hardcoded steps. Their integration keeps the system responsive to new input s
- Because of these libraries, building different parts of the system became smoother. Efficiency during development improved without extra effort.

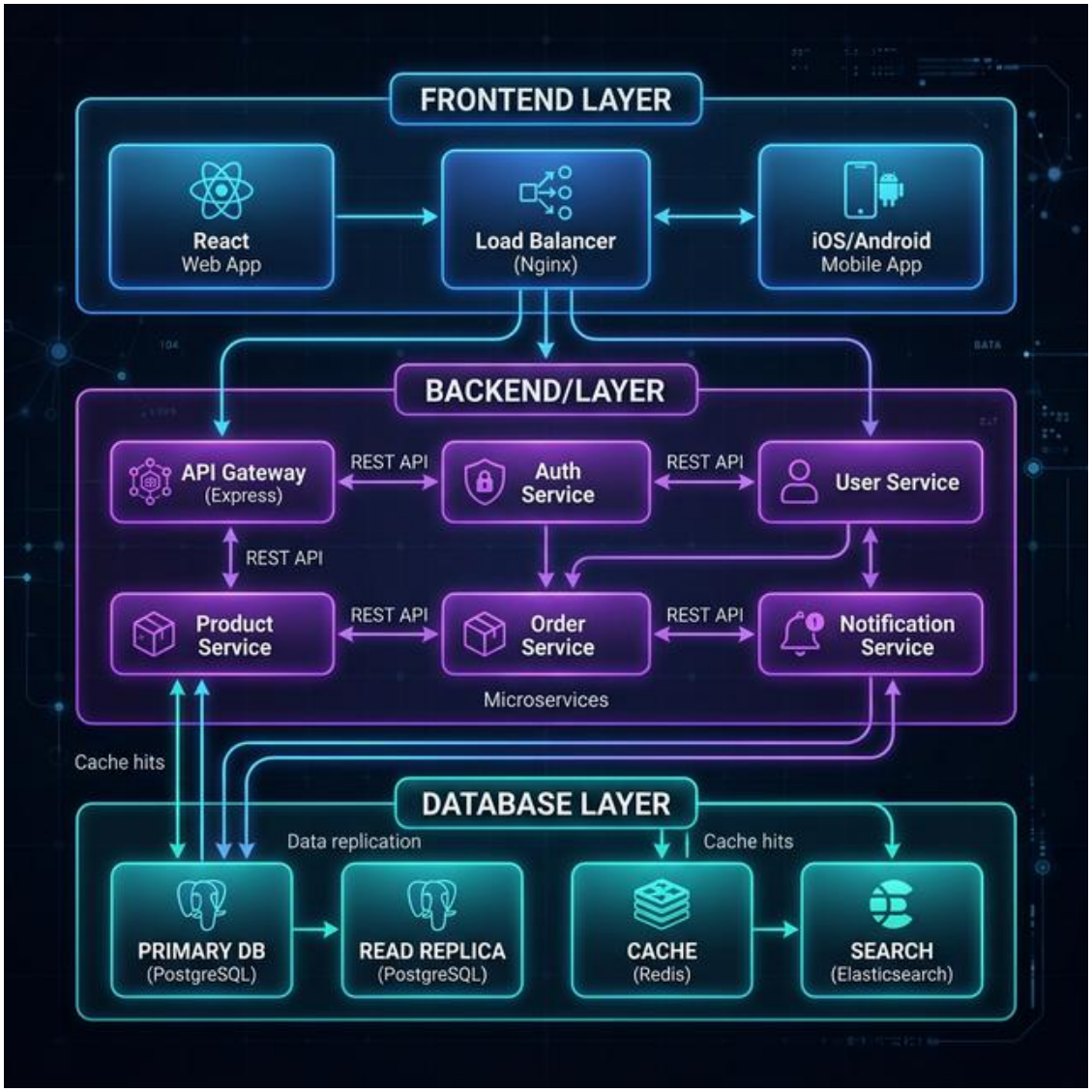
CHAPTER 5:

SYSTEM DESIGN

5.1 Overall Architecture

The College Event Aggregator follows a three-tier architecture with a clear separation between the presentation layer, the business logic layer, and the data layer. An additional intelligent services layer operates as an independent microservice, communicating with the business logic layer over HTTP.

Figure 5.1: Three-Tier System Architecture Diagram



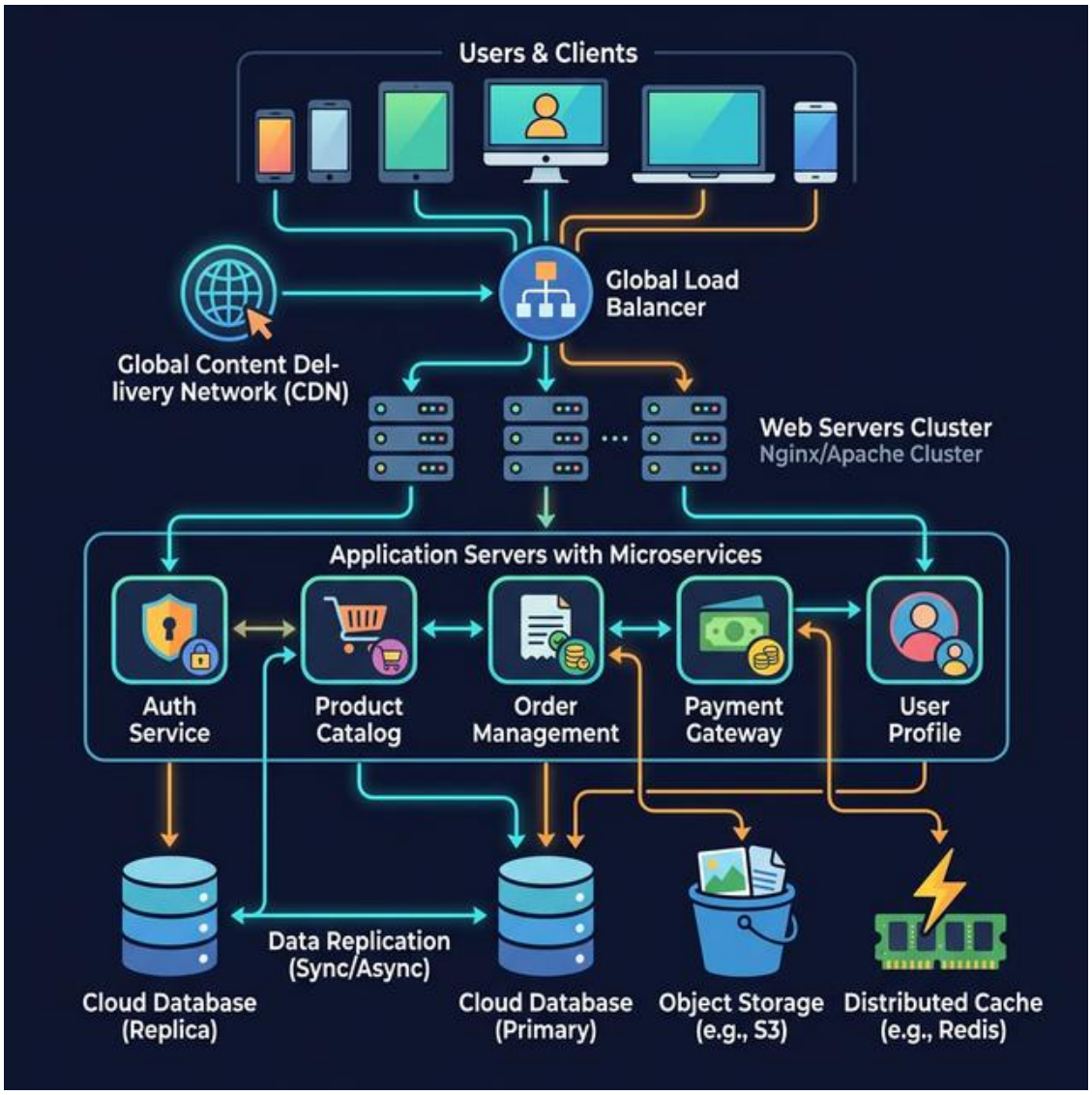
The request flow for a typical user action – say, a student searching for events – proceeds as follows:

- The student types a query in the React frontend search bar.
- The frontend sends a GET request to /api/v1/ai/search?q=upcoming+hackathon.
- The Express backend's AI routes module receives the request and forwards it to the FastAPI microservice at http://localhost:8000/search.
- The FastAPI microservice preprocesses the query (tokenization, stemming, synonym expansion), scores events against the processed query, and returns ranked results.

- The backend receives the results, optionally enriches them with full event data from PostgreSQL, and sends the response back to the frontend.
- The React frontend renders the search results as event cards.

5.2 Frontend Architecture

The frontend follows React's component-based architecture with a clear hierarchy:

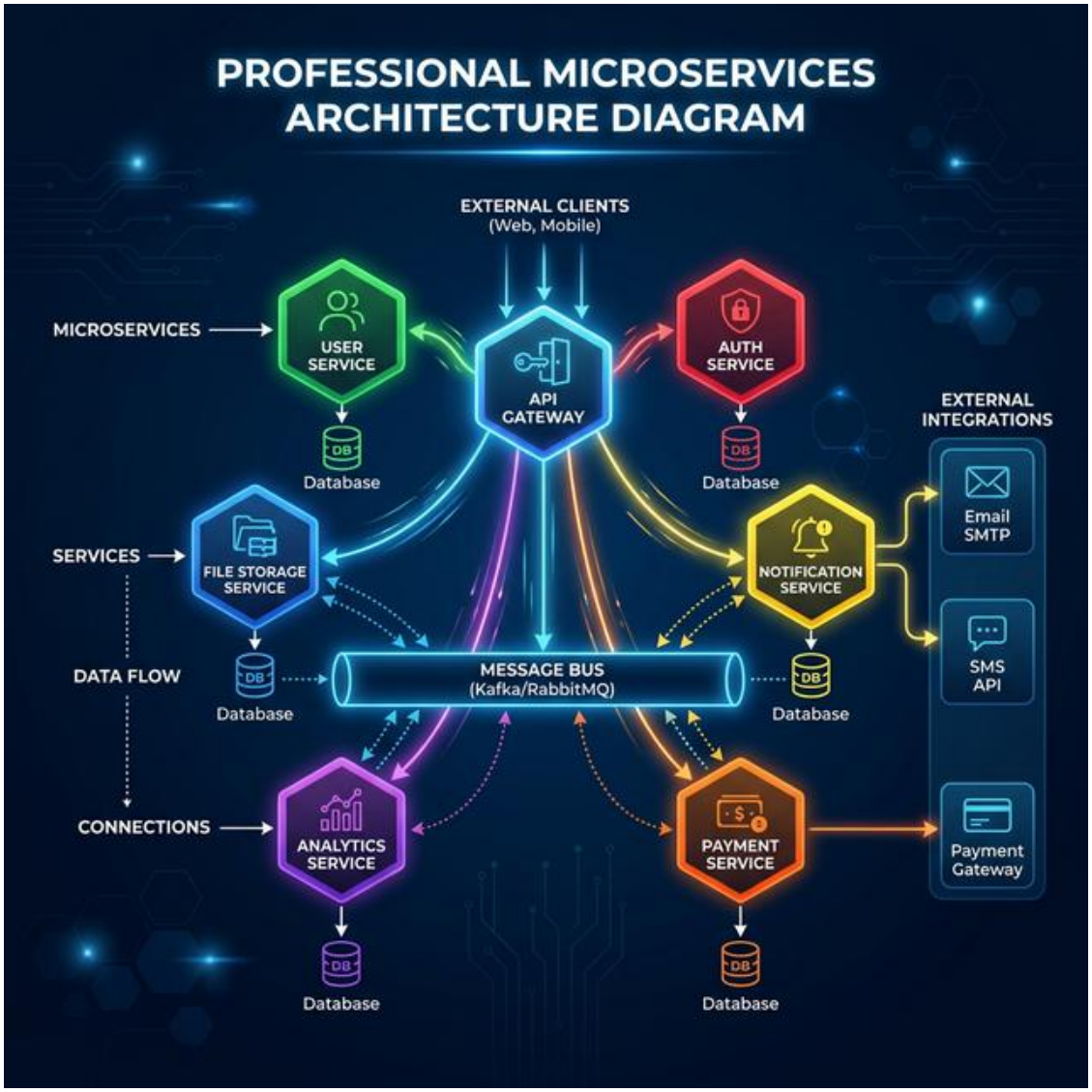


Key Design Decisions:

- **Context API over Redux:** Since the global state is limited to authentication and theme, React's built-in Context API was sufficient. Redux would have added unnecessary complexity.
- **API Client Class:** A centralized ApiClient class in lib/api.ts handles all HTTP communication, including automatic JWT header injection, consistent error handling, and typed responses.
- **Protected Routes:** A ProtectedRoute component wraps routes that require authentication and optionally checks user roles. Unauthorized users are redirected to the login page; users with insufficient roles are redirected to their dashboard.

5.3 Backend Architecture

The backend follows a layered architecture:



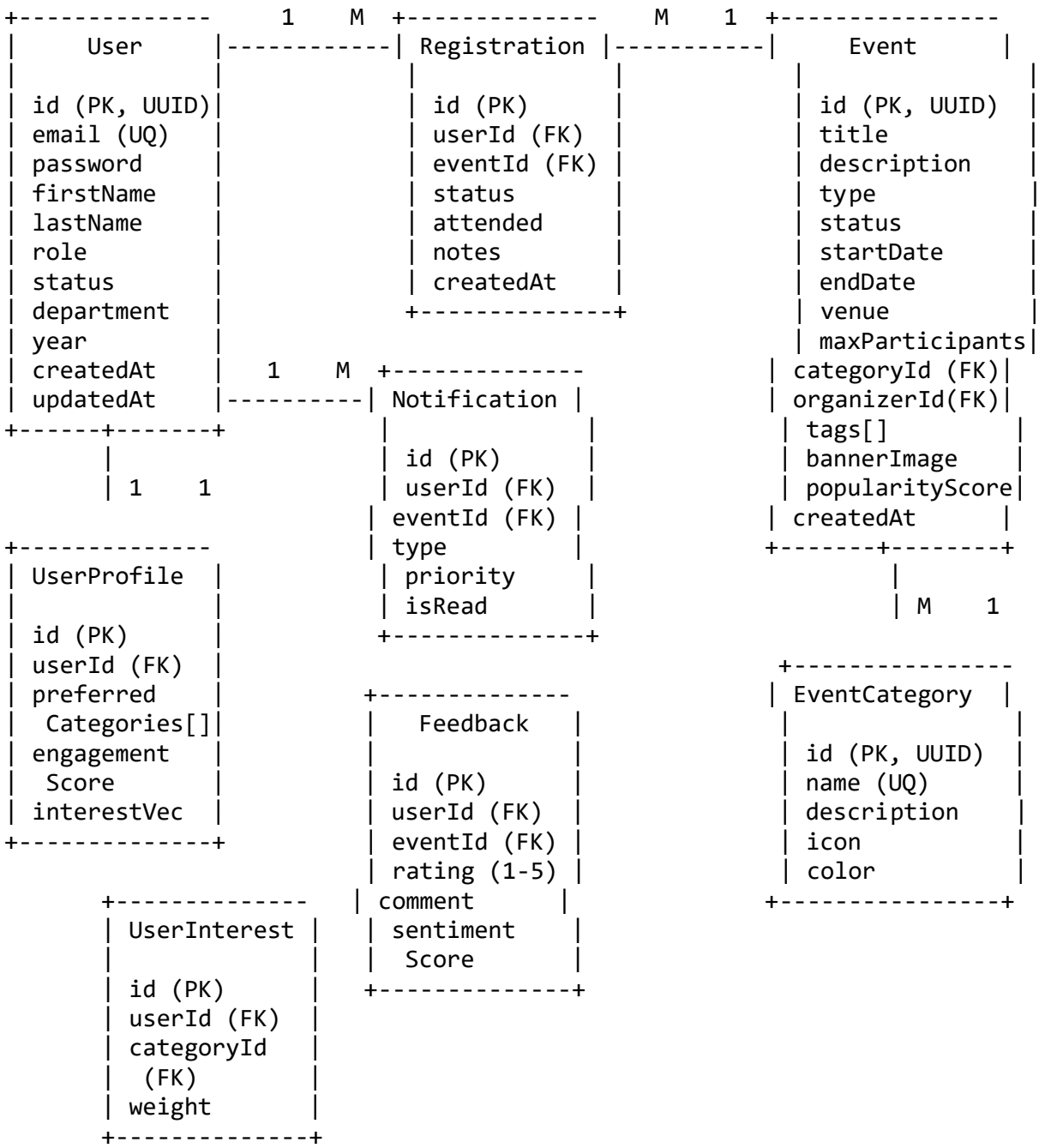
Request Processing Pipeline (server.ts):

- Helmet -> sets security headers
- CORS -> validates request origin
- Rate Limiter -> enforces request quotas
- Morgan -> logs HTTP request details
- Body Parsers -> parse JSON and URL-encoded bodies
- Route Handlers -> dispatch to appropriate controller
- Not Found Handler -> catches unmatched routes
- Error Handler -> formats and returns error responses

5.4 Database Design

5.4.1 Entity-Relationship Diagram

Figure 5.4: Entity-Relationship Diagram



5.4.2 Table Schemas

Table 5.1: Users Table Schema

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT uuid()	Unique user identifier

Column	Type	Constraints	Description
email	VARCHAR	UNIQUE, NOT NULL	Login email
password	VARCHAR	NOT NULL	bcrypt hashed password
firstName	VARCHAR	NOT NULL	User's first name
lastName	VARCHAR	NOT NULL	User's last name
role	ENUM	DEFAULT 'STUDENT'	STUDENT/ORGANIZER/FACULTY/ADMIN
status	ENUM	DEFAULT 'ACTIVE'	ACTIVE/INACTIVE/SUSPENDED
department	VARCHAR	NULLABLE	Academic department
year	INT	NULLABLE	Year of study (students)
createdAt	TIMESTAMP	DEFAULT now()	Account creation time
updatedAt	TIMESTAMP	Auto-updated	Last modification time
lastLogin	TIMESTAMP	NULLABLE	Most recent login

Table 5.2: Events Table Schema

Column	Type	Constraints	Description
id	UUID	PK	Unique event identifier
title	VARCHAR	NOT NULL	Event name
description	TEXT	NOT NULL	Full description
type	ENUM	NOT NULL	ACADEMIC/CULTURAL/SPORTS/TECHNICAL/etc.
status	ENUM	DEFAULT 'DRAFT'	DRAFT/PENDING/APPROVED/REJECTED/etc.
startDate	TIMESTAMP	NOT NULL	Event start

Column	Type	Constraints	Description
endDate	TIMESTAMP	NOT NULL	Event end
venue	VARCHAR	NOT NULL	Location
maxParticipants	INT	NULLABLE	Capacity limit
categoryId	UUID	FK -> event_categories	Event category
organizerId	UUID	FK -> users	Event creator
tags	TEXT[]	DEFAULT []	Searchable tags
bannerImage	VARCHAR	NULLABLE	Image URL
popularityScore	FLOAT	DEFAULT 0	Computed popularity

Table 5.3: Registrations Table Schema

Column	Type	Constraints	Description
id	UUID	PK	Registration identifier
userId	UUID	FK -> users, CASCADE	Registering user
eventId	UUID	FK -> events, CASCADE	Target event
status	ENUM	DEFAULT 'REGISTERED'	Registration status
attended	BOOLEAN	DEFAULT false	Attendance flag
(userId, eventId)	–	UNIQUE	One registration per user per event

5.5 Data Flow Diagrams

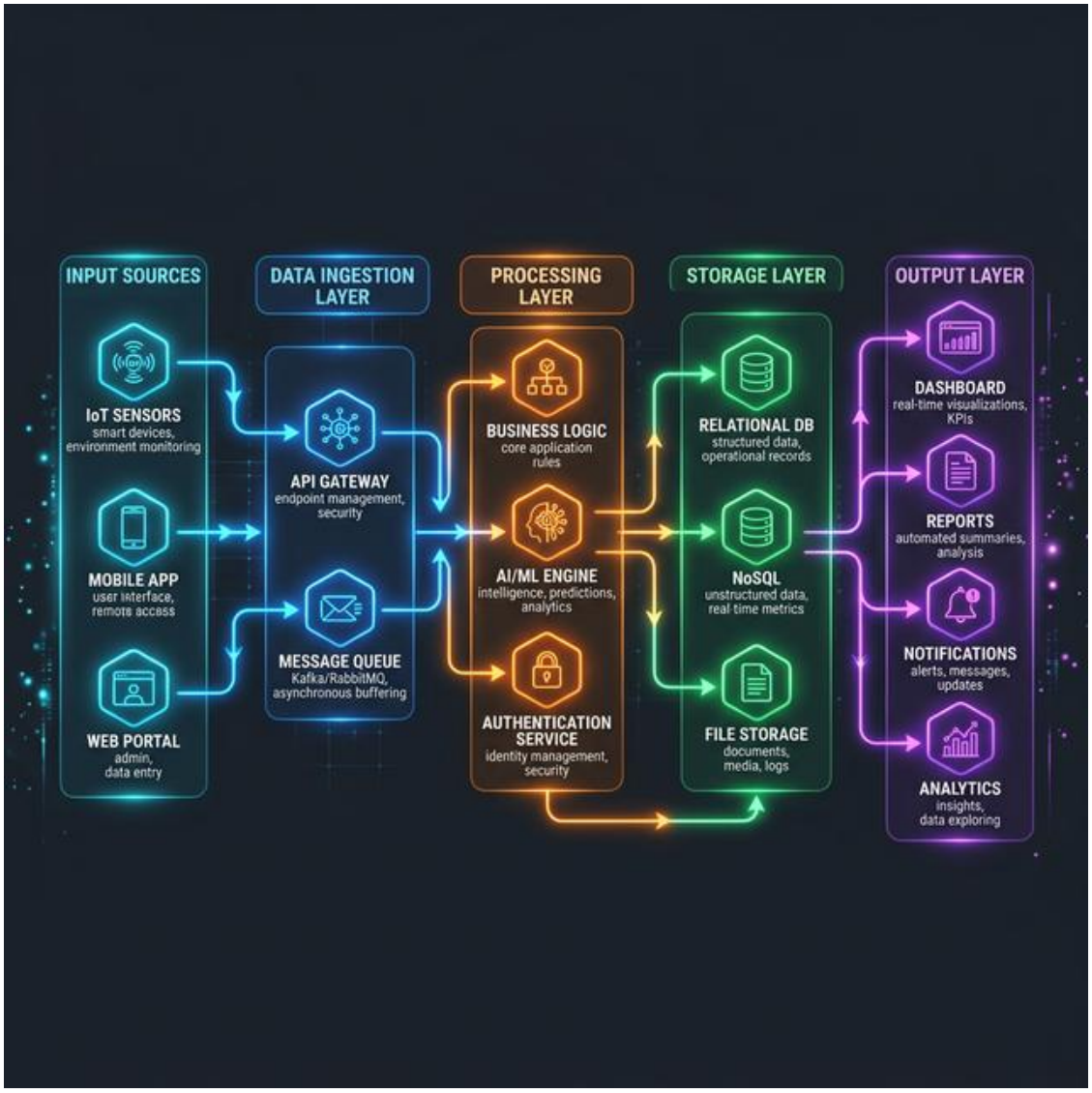
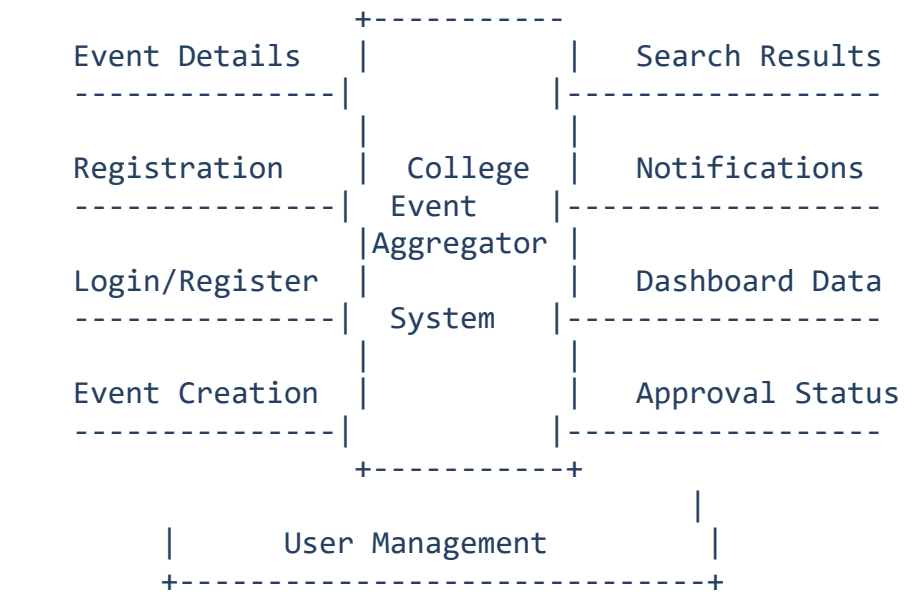


Figure 5.5: Data Flow Architecture

DFD Level 0 – Context Diagram

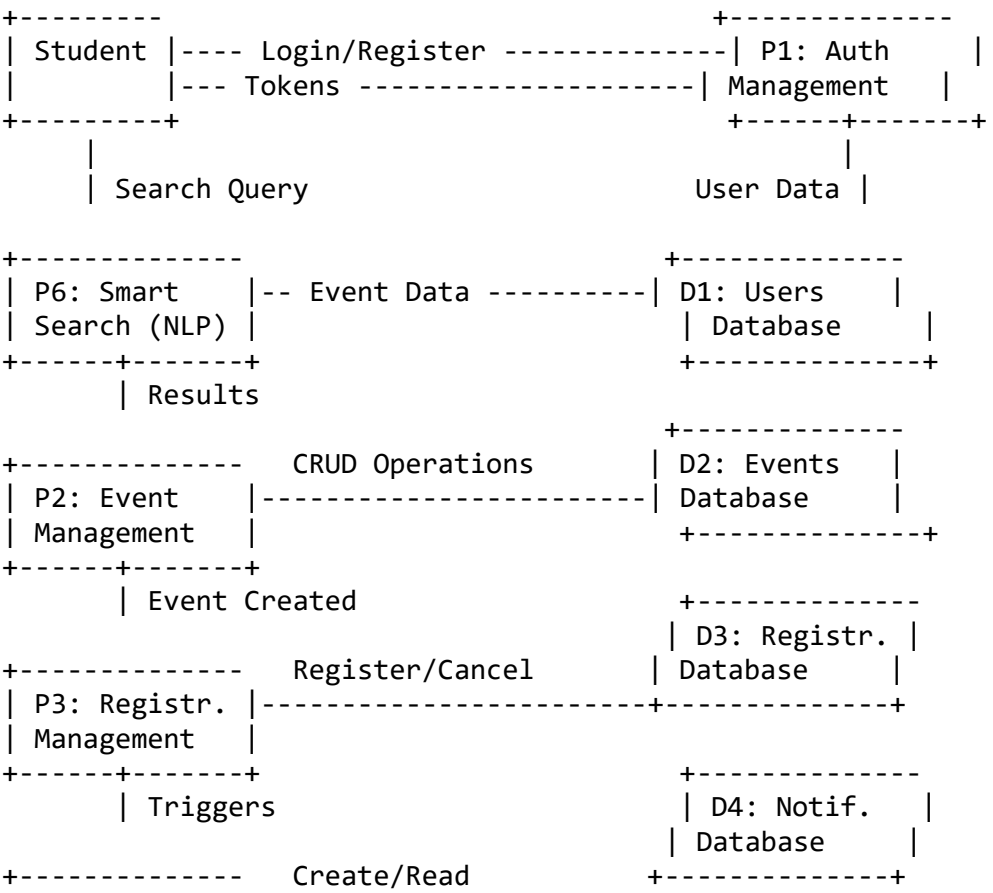
Figure 5.5: Context Diagram



DFD Level 1 – Process Decomposition

Figure 5.6: DFD Level 1

Figure 5.6: DFD Level 1



```
| P4: Notific. |-----------------------
| System       |                              |
+--------------+                              |
                                              |
+--------------   Approve/Reject              |
| P5: Approval |----- Status Update ------------+
| Workflow     |
+--------------+

+--------------
| P7: Recommend|-- User Preferences + Event Data
| Engine       |---- Scored Recommendations ---- Student
+--------------+
```

5.6 Use Case Diagram

Figure 5.7: Use Case Diagram

```
+-------------------------------------------------------------------
|                    College Event Aggregator                        |
|                                                                      |
|  +-------------     +-------------------                          |
|  |  Register   |     |  Login / Logout   |                          |
|  +------+------+     +--------+----------+                          |
|         |                     |                                     |
|  +------------     +------------------                             |
|  |Browse Events|     | View Dashboard    |                          |
|  +------+------+     +--------+----------+                          |
|         |                     |                                     |
|  +------------     +------------------                             |
|  |Search Events|     |View Notifications |                          |
|  +------+------+     +-------------------+                          |
|         |                                                           |
|  +----------------                                                  |
|  |Register for     |                                                 |
|  |Event            |--- Student, Organizer, Faculty, Admin           |
|  +-----------------+                                                 |
|                                                                      |
|  +-----------------                                                 |
|  | Create Event    |--- Organizer, Faculty, Admin                    |
|  +--------+--------+                                                 |
|           |                                                          |
|  +----------------                                                  |
|  | Edit/Delete     |                                                 |
|  | Own Events      |--- Organizer, Faculty, Admin                    |
|  +-----------------+                                                 |
|                                                                      |
|  +-----------------                                                 |
|  | Approve/Reject  |                                                 |
|  | Events          |--- Faculty, Admin                               |
|  +-----------------+                                                 |
|                                                                      |
|  +----------------  +-----------------                              |
|  | Manage Users    |  | View System     |                          |
|  |                 |  | Statistics      |--- Admin Only            |
```

```
|  +----------------+  +----------------+                    |
+-------------------------------------------------------------+
```

5.7 Sequence Diagrams

Figure 5.8: Sequence Diagram – User Registration

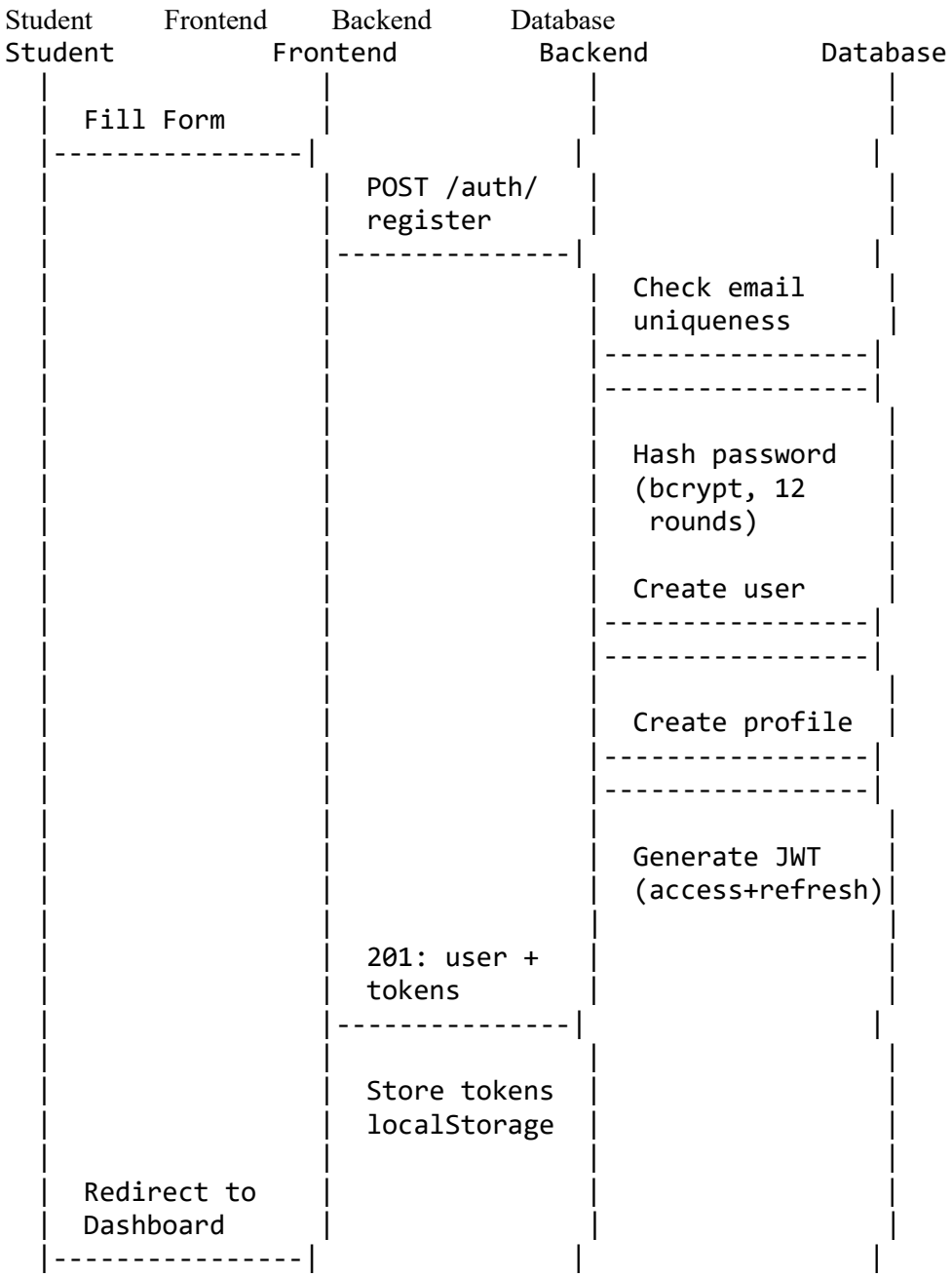
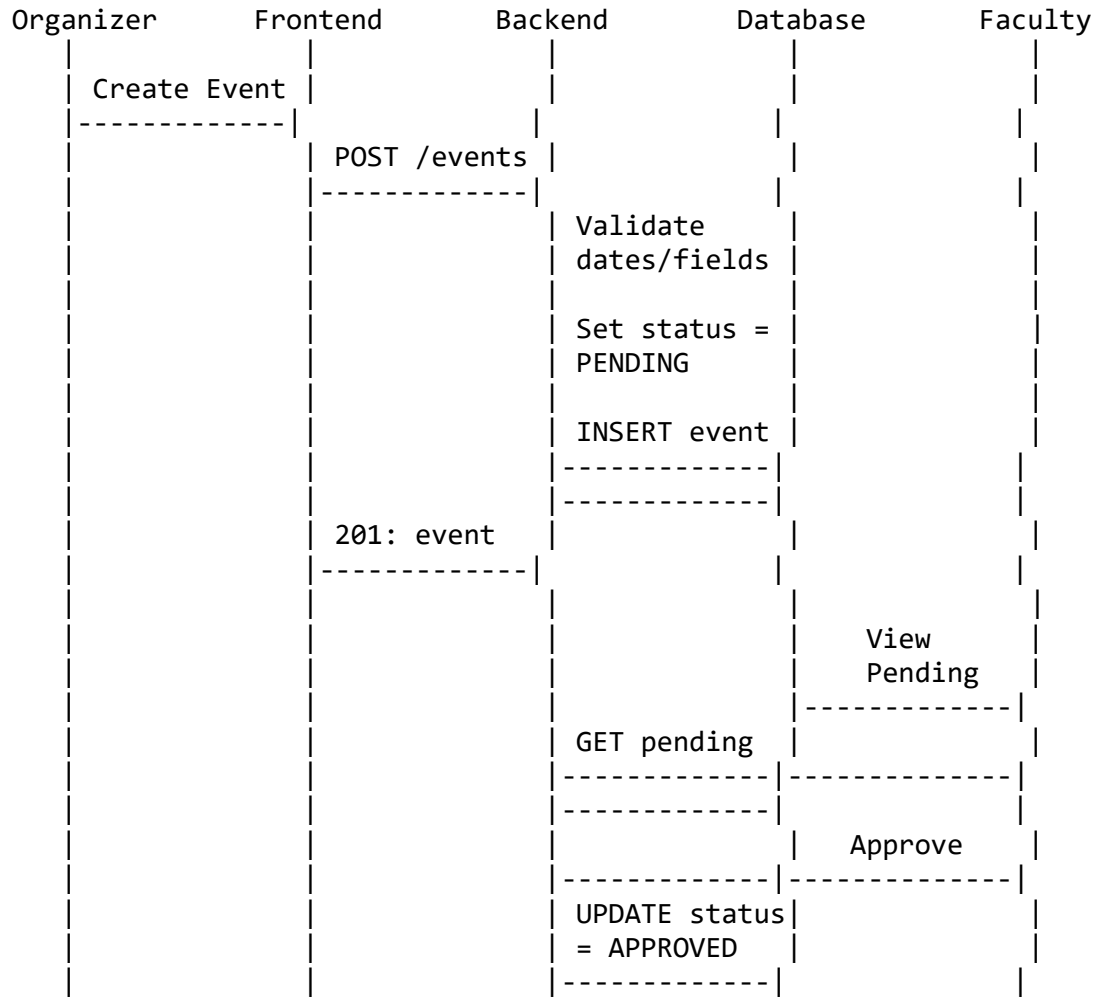
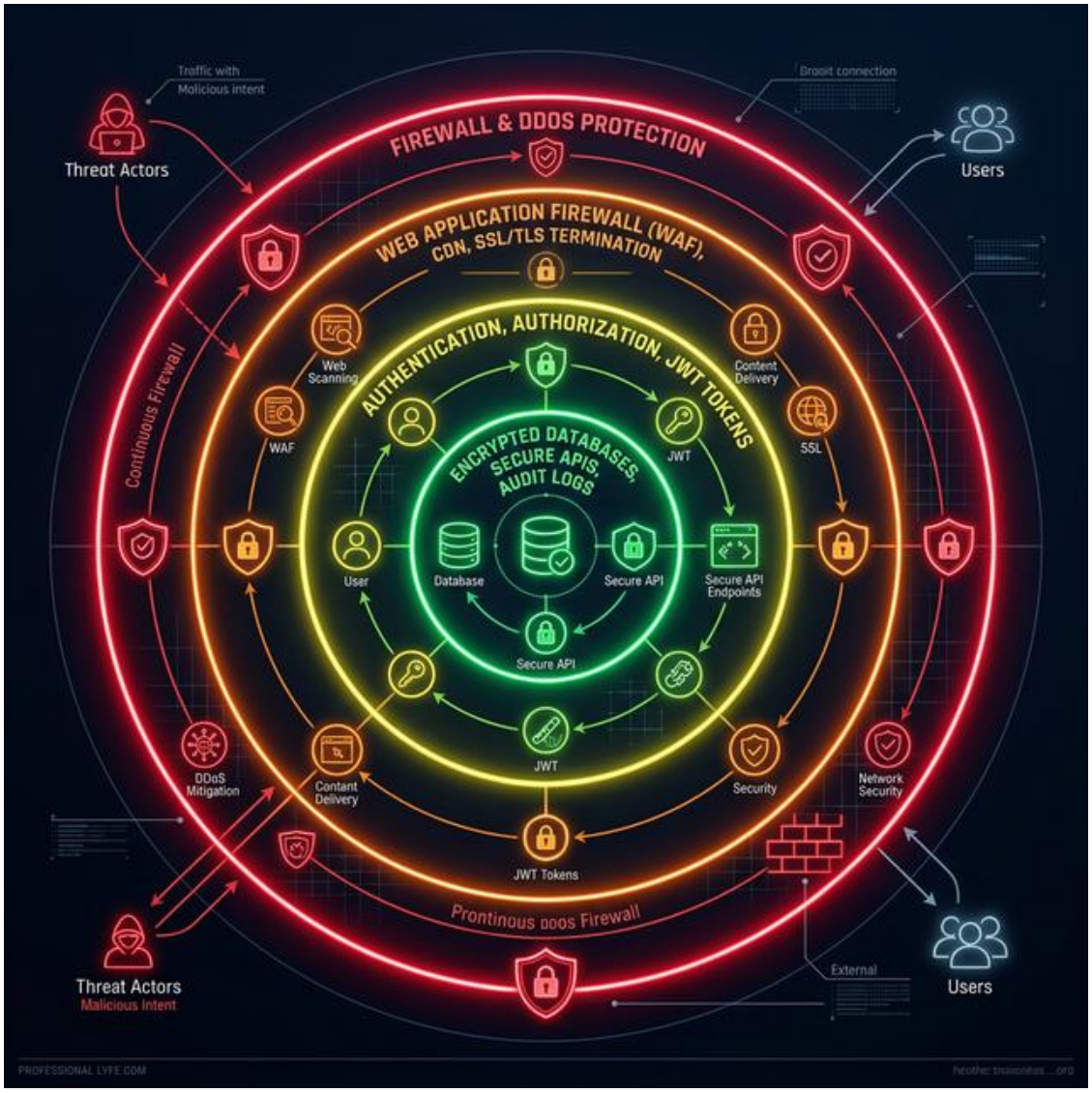


Figure 5.9: Sequence Diagram – Event Creation and Approval



5.8 Security Design

Figure 5.10: JWT Authentication Flow



Security Layers Implemented:

- **Helmet.js** – Sets HTTP security headers (Content-Security-Policy, X-Content-Type-Options, X-Frame-Options)
- **CORS** – Restricts API access to authorized frontend origins
- **Rate Limiting** – 100 requests per 15 minutes (general), 5 per 15 minutes (auth endpoints)
- **JWT Authentication** – Short-lived access tokens with refresh rotation
- **bcrypt Hashing** – Cost factor of 12 for password storage
- **Input Validation** – express-validator on all form submissions
- **Role-Based Authorization** – Middleware checks user roles before granting access

CHAPTER 6:

DETAILED IMPLEMENTATION

6.1 Authentication Module

Inside every entry point sits the authentication piece. This part manages signing up, logging in, updating tokens, also changing passwords.

Right after someone signs up, the system jumps into action. A fresh sign-up lands in auth.controller.ts for handling. Once data arrives, it gets checked right away. The moment validation passes, an account springs to life. No delays - storage happens just as details clear inspection. From start to finish, each move follows a quiet rhythm. Submission triggers a chain that runs without pause. Every step clicks forward only when the prior one settles. Flow stays tight, never straying from its path. Process ends where confirmation begins

Check every form field by applying express-validator checks - like correct email structure, passwords needing eight characters, plus making sure first and last names aren't missing.

Beyond just looking up an email, the system first scans its records. A match means someone with that address is already inside. If found, nothing new happens - just recognition of what's there. Otherwise, space opens for something fresh. That moment shapes next steps without force or drama.

Built-in delays slow down guessers when turning passwords into unreadable strings through bcrypt at level twelve. This method spends about three hundred milliseconds on each conversion, blocking rapid break-in tries by stretching every attempt. Slowness here acts like a shield, so trying thousands of options becomes too time-consuming.

4.Create the user record in the users table via Prisma.

Built a linked user_profiles entry to hold choices about recommendations. This one keeps track of what each person likes. Information goes here instead of elsewhere. Set it up so updates happen smoothly. Made sure it connects properly to the main account.

A fresh set of tokens appears - access one sealed with JWT_SECRET, good for fifteen minutes. The second piece follows different rules: signed using JWT_REFRESH_SECRET, lasts seven full days. One opens the door now. The other waits in reserve, ready when time runs out.

Send back the user information along with tokens to the front end.

When someone tries to sign in, the system first checks their details. If the account is marked SUSPENDED or not ACTIVE, access gets denied right away. Instead of just confirming passwords, it runs a secure match between what's typed and the encrypted version saved earlier. Once verified, time of entry gets recorded fresh. Tokens are created only after everything else passes.

Midway through the session, the front end reaches out to /auth/refresh when the access token runs out. Because the system checks both digital signature and expiration date of the refresh token, trust stays intact. Once confirmed, the server looks up whether the person still has an active account. Only then does it hand back a brand-new set of tokens. Every time this happens, older refresh credentials get retired on the spot - cutting off potential misuse before it spreads.

6.2 Event Management Module

A single file, event.controller.ts, manages every stage of events - making it the trickiest part of the system. This piece does more than others by far, guiding events from start to finish.

When someone makes a new event via POST /events, the system first examines the incoming data. It confirms the start time comes earlier than the ending time - no exceptions. Depending on who's creating it, the starting state changes: ORGANIZER entries go into PENDING_APPROVAL. But if the user is

FACULTY or ADMIN, approval happens right away - no waiting. Everything gets saved - the full details, tags tucked neatly as a PostgreSQL array, plus space for the banner picture link. Once stored, that record lives in the database just like the others.

Listing events via GET /events comes with smart filters built in. Depending on the given query inputs, the controller shapes a custom Prisma where condition. Each parameter tweaks how data gets pulled

Approval state shapes what appears - set it to show only approved items unless needed otherwise. Different views start with approved as the go-to choice. Change this rule if another status fits better. What you pick here decides visibility at a glance

Choose events using category like ACADEMIC or CULTURAL. Pick one kind at a time instead of mixing types. Filtering helps spot what matches your interest. Options show only when selected type fits. Different kinds appear once you switch filters. Look through each group separately for clarity

·categoryId – filter by event category

·search – text search across title, description, and tags

·isUpcoming – upcoming events only

·isOngoing – currently running events

·Date range filters for start dates

·Pagination with configurable page size

Whatever column you pick, tap it to flip the sort direction. One click climbs alphabetically, next reverses. Each header becomes a toggle - no extra menus. Direction shifts every time without prompts. Flip back instantly whenever needed

Apart from the event details, you also get info on how many results exist overall. Total page numbers show up alongside indicators for whether more pages wait ahead. Previous ones might be accessible too - this depends on your current spot in the list. Everything arrives together when the system replies.

Approval happens through a PUT request at /events/:id/approve. Only faculty or admin accounts hold permission to move forward. If the system finds the event missing, nothing changes. When present, its current state must be PENDING_APPROVAL - no exceptions. Once confirmed, the status shifts directly to APPROVED. A timestamp marks the exact moment of confirmation. The user responsible gets logged alongside that time. Every detail locks in place after update. Nothing proceeds without proper clearance first.

Putting a stop to an event through PUT /events/:id/reject works much like approving one - yet here, the system keeps track of why the reviewer said no. That explanation then shows up for the organizer to see. Instead of moving forward, the request gets turned aside with clear notes on what blocked it.

6.3 Registration Module

Every so often, someone signs up through the system that links people to activities. Handling these signups means tracking who joined what, making sure details match, while updating records quietly behind the scenes

Happens when someone signs up. Checks if the event is real, given the okay, still open for signups, hasn't filled all spots. Makes sure the person isn't signing up twice. If everything fits, it logs a new entry marked as REGISTERED.

Cancelling registration means students can remove their sign-up themselves. Their status shifts to CANCELLED once done. This update happens right after they choose to opt out. A student's record

reflects the change immediately. No approval is needed for this step. The system processes it without delays. Status appears as CANCELLED across all views. Students manage this anytime before deadlines pass.

Check your sign-ups here. See everything tied to your account. Filter by status if you want. Results show only what belongs to you. Pick which states appear, or skip that step. Everything listed matches your profile. Only active choices display unless you say otherwise.

With this tool, event hosts see who signed up. Attendance details get adjusted right there. Waitlists are handled the same way. People on hold can move in when spots open. Everything stays updated live.

6.4 Dashboard Module

Inside dashboard.controller.ts, three dashboards appear - each shaped for a particular role. One loads for admins, another kicks in for managers, the last serves staff. Each view adjusts silently based on who logs in

What shows up here? All sign-ups added together, how many gatherings were actually joined, what is booked next, how many alerts wait unread, a lineup of coming events holding a spot, suggestions shaped just for them behind the scenes.

From here, event totals appear at a glance - along with how many sit pending, approved, or turned down. Registration numbers stack up across every gathering, no matter the type. Attendance rates get worked out automatically, based on real sign-ins. Lately held happenings show just below, each tagged with who signed up. Coming soon? A row of next steps floats above the rest, counting spots claimed so far.

Numbers across the platform show up front here. Total people using the system appears right away, followed by how many joined recently. Events created so far fill another slot, while waiting requests sit nearby. What is starting soon shows just above what is already running. Sign ups overall get their own spot too. A line chart hints at whether user numbers are climbing or dipping. Pie slices reveal where events stand - active, finished, or still in queue.

6.5 Notification Module

Each time something happens inside the system, a notification shows up on its own. When certain actions occur, alerts appear without anyone needing to start them. The moment an event triggers, the software sends out a message instantly. Following key changes, updates pop up automatically across the interface. As activity meets set conditions, signals get pushed to users right away

·Registration confirmation when a student signs up

Folks running events will get messages about whether their plans are cleared or turned down

·Event reminders as deadlines approach

Fresh updates pop up when big happenings gain attention

Running on a timer set by node-cron, the notification.service.ts scans for upcoming events needing alerts. A trigger fires at intervals, checking which reminders should go out next. Each alert carries details: kind of notice, headline, body text, urgency rank, plus a possible connection to the actual event. Scheduled tasks handle timing - no manual push needed to start. Information flows automatically when conditions match predefined rules. Details appear only if an event meets criteria tied to time and importance. Linking back happens selectively, depending on context relevance.

6.6 Recommendation Engine

A twist on tradition lives inside ai-service/src/services/recommendation.py - its suggestion system blends three methods at once. One part leans on patterns found in user behavior. Another taps into item similarities, working quietly behind the scenes. A third piece mixes historical results with live choices, stitching them together without fanfare. Together - they form a combined logic that adapts as it moves

1. Half the ranking comes from matching what you like with an event's type. A match gives 0.7 points right away. Your past activity can push that higher by up to 0.2. How many others join affects it too - another 0.1 at most. Only those hitting over 0.3 make the list shown to you.

2. Starting off, events land on the list if their total rating clears 0.4. Their spot depends mostly on how well-known they are - six-tenths of that mark comes from public interest. A bit less weight goes to current momentum, making up four-tenths. This whole ranking carries a third of the overall value.

3. Soon matters most. Events under two weeks away get top marks. A countdown shapes the score - closer dates rise higher. Each day closer lifts importance. Fourteen-day windows define urgency. Points drop as delays grow. Immediacy earns value. The math is simple: one minus delay divided by fourteen. Nearness decides rank. What happens fast gets weight.

Out of nowhere, overlapping entries vanish when those three lists come together. Weighted scores take charge after duplicates get tossed aside. Sorted tight, only the highest scorers make the cut - top N stays standing.

6.7 Smart Search Engine

The search service in ai-service/src/services/search.py processes queries through a five-step pipeline:

Step 1: Preprocessing. The raw query is converted to lowercase, special characters are removed, the text is tokenized using NLTK's word_tokenize, stopwords (including custom additions like "event", "events", "college") are removed, and remaining tokens are stemmed using the Porter Stemmer.

Step 2: Synonym Expansion. Each stemmed token is checked against a synonym dictionary. For example, if the query contains "hackathon", the expanded terms include "contest", "tournament", "challenge", and "competition" (all stemmed).

Step 3: Intent Detection. The original query is scanned for intent keywords. For example, "upcoming" triggers the find_upcoming intent, "popular" triggers find_popular. The detected intent influences the final scoring.

Step 4: Relevance Scoring. Each event is scored against the expanded query using four factors: title match (40% weight), description match (30%), category match (20%), and tag match (10%). Intent-aligned events receive a multiplicative boost (1.2x for upcoming, 1.3x for popular).

Step 5: Suggestion Generation. Based on the query and results, the service generates alternative search suggestions (e.g., adding "workshop" or "upcoming" to the query).

6.8 Admin Panel

The admin panel provides three pages:

Admin Dashboard (AdminDashboardPage.tsx): Displays system-wide cards with key metrics and charts showing user growth and event distribution.

Pending Approvals (PendingApprovalsPage.tsx): Lists events awaiting approval with event details, organizer information, and approve/reject action buttons.

User Management (UserManagementPage.tsx): Displays all users in a searchable, filterable table with actions to change roles, suspend accounts, or delete users.

6.9 Error Handling and Logging

Backend Error Handling (error.middleware.ts): A global error handler middleware catches all errors thrown in controllers and formats them into a consistent response structure: { success: false, message: string, error: { code: string, details?: any } }. Custom error classes (BadRequestError, UnauthorizedError, NotFoundError, ForbiddenError, ConflictError) map to appropriate HTTP status codes.

Logging (utils/logger.ts): The Winston logging library is configured with timestamped entries and multiple log levels (error, warn, info, debug). In development, logs output to the console with colorized formatting. In production, logs would be written to files with rotation policies.

Frontend Error Handling: The API client class catches fetch errors and surfaces them through toast notifications (using the Sonner library). Network errors, authentication failures, and validation errors are all handled with user-friendly messages.

CHAPTER 7:

PROJECT MODULES

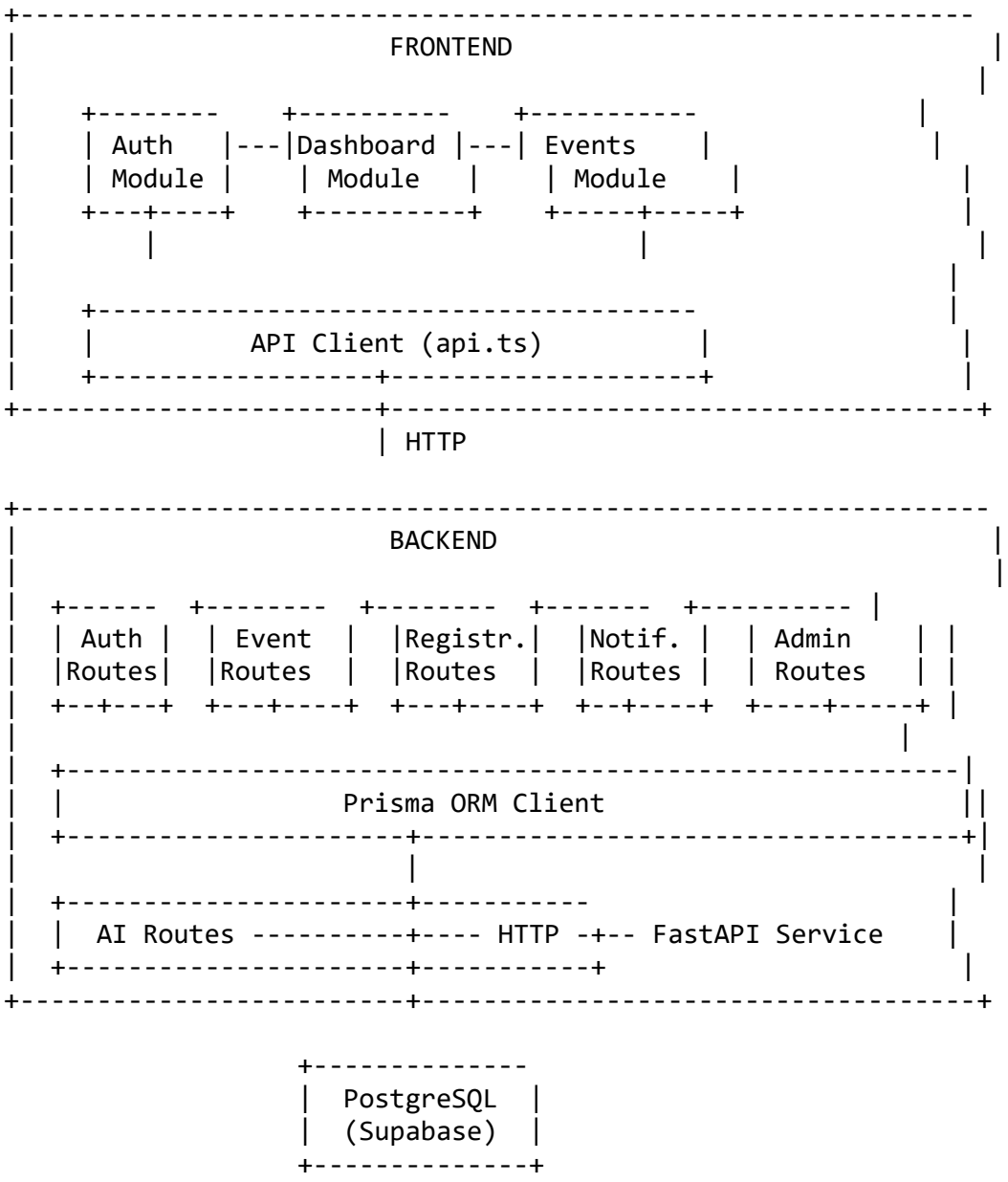
7.1 Module Overview

Table 7.1: Module Overview

Module	Files	Layer	Description
Authentication	auth.controller.ts, auth.routes.ts, auth.middleware.ts, useAuth.tsx, LoginPage.tsx, RegisterPage.tsx	Full Stack	User registration, login, JWT management
Event Management	event.controller.ts, event.routes.ts, EventsPage.tsx, EventDetailPage.tsx, CreateEventPage.tsx	Full Stack	Event CRUD, filtering, pagination
Registration	registration.controller.ts, registration.routes.ts, MyRegistrationsPage.tsx	Full Stack	Event sign-up, cancellation, waitlisting
Notifications	notification.controller.ts, notification.routes.ts, notification.service.ts, NotificationsPage.tsx	Full Stack	Alert generation, delivery, management
Dashboards	dashboard.controller.ts, dashboard.routes.ts, DashboardPage.tsx	Full Stack	Role-specific analytics views
Admin Panel	admin.controller.ts, admin.routes.ts, AdminDashboardPage.tsx, PendingApprovalsPage.tsx, UserManagementPage.tsx	Full Stack	User/event management, system stats
Recommendation Engine	recommendation.py, /recommendations endpoint	AI Service	Hybrid event recommendations
Smart Search	search.py, /search endpoint	AI Service	NLP-powered event search
Category Predictor	categorization.py, /predict-category endpoint	AI Service	Automatic event categorization
Notification Scorer	notification_scorer.py, /notification-priority endpoint	AI Service	Priority scoring for alerts
API Client	api.ts	Frontend	Centralized HTTP communication
Theme System	useTheme.tsx	Frontend	Dark/light mode management

7.2 Module Interaction Diagram

Figure 7.1: Module Interaction Diagram



7.3 User Flow

Student Flow:

To get started you need to sign up or log in. When you are inside the dashboard will show you what is coming soon. It will also show you some smart picks. You can find things by browsing or by typing what you are looking for. When you click on an event it will open up. You will see all the information about it. If you want to sign up for an event it only takes a steps. After you sign up everything else happens automatically. You will get a message that says you are in. Then you can look at the events you signed up for. When it is time, for the event you just need to show up. After the event you can share what you thought about it.

Organizer Flow:

The first thing you need to do is sign in. After you sign in you will see the organizer dashboard so take a look. Next you need to set up an event and you can do that by filling out the form. When you are done with the form you can send it in for approval. Later you can check back to see what is happening with your event. Now the last step is still open. You can look at who signed up for your events that were approved. Then you need to take care of the list of people who are coming. Finally you can see how your event did overall.

Admin Flow:

You start by going to the login screen. When you log in the admin dashboard will show you the system numbers right away. Then you will see the events that are waiting for approval so you can look at them. Some events will be approved and some will not be approved it just depends on what you think. After that you can manage the users, which means you can change what they can do. If they are active or not. You need to keep an eye on the system the time you are logged in.

CHAPTER 8:

SYSTEM TESTING

8.1 Testing Strategy

We tested the system step by step. We started with parts and then put them together to see how they work as a whole system. Each part was tested on its own before we tested how it works with parts. Here are the main ways we tested the system:

1. Manual Functional Testing – We checked each feature manually using the user interface to make sure it works as expected. We tested the system manually.

2. API Testing – We used Postman and cURL to test the backend. We made sure it gives responses and handles errors correctly. We tested API.

3. Cross-Browser Testing – We tested the frontend on browsers like Chrome, Firefox and Edge. We made sure it looks and works the same on all browsers. We tested browsers.

4. Responsive Testing – We checked the user interface on screen sizes. We made sure it adjusts correctly. We tested the interface.

8.2 Types of Testing

Unit Testing: We tested each part of the system on its own. We tested backend parts with data and frontend parts separately. We made sure they work correctly. We tested units.

Integration Testing: We tested how different parts of the system work together from start to finish. For example we tested event creation from start, to finish. We made sure it works correctly. We tested integration.

System Testing: We tested the system with all its parts working together. We checked user flows like registering for an event and getting notifications. We tested the system.

Security Testing: We tried to break into the system. We tested security measures. We checked if the system can stop people from getting in without permission and handle requests. We tested security.

Usability Testing: We asked students to use the system and give feedback on how easy it's to use. We asked them to do tasks like finding and registering for an event. We used their feedback to make the system better. We tested usability.

8.3 Test Cases

Table 8.1: Authentication Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-01	Valid registration	Valid email, password (8+ chars), name	201, user + tokens	Pass

TC ID	Test Case	Input	Expected Output	Result
TC-02	Duplicate email registration	Existing email	409, "User already exists"	Pass
TC-03	Invalid email format	"notanemail"	400, validation error	Pass
TC-04	Short password	"123"	400, "Password too short"	Pass
TC-05	Valid login	Correct email + password	200, user + tokens	Pass
TC-06	Wrong password	Correct email, wrong password	401, "Invalid credentials"	Pass
TC-07	Login with suspended account	Suspended user credentials	401, "Account suspended"	Pass
TC-08	Token refresh	Valid refresh token	200, new token pair	Pass
TC-09	Expired refresh token	Expired token	401, "Token expired"	Pass
TC-10	Change password	Correct current, valid new	200, "Password changed"	Pass

Table 8.2: Event Management Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-11	Create event (Organizer)	Valid event data	201, status=PENDING_APPROVAL	Pass
TC-12	Create event (Admin)	Valid event data	201, status=APPROVED	Pass
TC-13	Invalid date range	endDate before startDate	400, "End date must be after start"	Pass
TC-14	List approved events	No auth required	200, array of events	Pass
TC-15	Filter by category	categoryId param	200, filtered events	Pass
TC-16	Search events	search="hackathon"	200, matching events	Pass

TC ID	Test Case	Input	Expected Output	Result
TC-17	Approve event (Faculty)	Event in PENDING status	200, status=APPROVED	Pass
TC-18	Approve already-approved	Event in APPROVED status	400, "Not pending"	Pass
TC-19	Delete event (non-owner)	Different user's event	403, "No permission"	Pass
TC-20	Pagination	page=2, limit=5	200, correct page metadata	Pass

Table 8.3: Registration Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-21	Register for event	Valid eventId	201, registration record	Pass
TC-22	Duplicate registration	Already registered event	409, "Already registered"	Pass
TC-23	Register for full event	Event at capacity	400, "Event is full"	Pass
TC-24	Cancel registration	Own registration ID	200, status=CANCELLED	Pass
TC-25	View my registrations	Authenticated user	200, registration list	Pass

Table 8.4: Search and Recommendation Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-26	NLP search with synonyms	"coding contest"	Results include "hackathon" events	Pass
TC-27	Intent detection	"upcoming workshops"	find_upcoming intent detected	Pass
TC-28	Recommendations for new user	No preferences set	Popularity-based results	Pass
TC-29	Recommendations with prefs	preferred_categories set	Content-based results prioritized	Pass
TC-30	Category prediction	title + description	Predicted category with confidence	Pass

Table 8.5: Admin Panel Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-31	Access admin as Student	Student JWT	403, "Forbidden"	Pass
TC-32	Change user role	userId, new role	200, updated user	Pass
TC-33	Suspend user	userId, status=SUSPENDED	200, user suspended	Pass
TC-34	View system statistics	Admin JWT	200, stats object	Pass
TC-35	Delete user	userId	200, user deleted	Pass

8.4 Results and Observations

We got some results from our tests. Here are the key things we found out:

- The rate limiter worked as expected. It blocked requests after a number and returned a 429 status code.
- The JWT refresh flow was smooth. The frontend spotted expired access tokens. Refreshed them automatically. This did not interrupt the user experience.
- Our search engine did a job. For example when users searched for "coding contest" they also got results tagged with "hackathon" and "programming".
- We tested our pages on screen sizes. They worked well on screens, like the iPhone SE (320px) and big screens, like ultra-wide desktops (2560px).
- We also tested on browsers. Our pages looked good on Chrome, Firefox and Edge. There were no rendering issues.

8.5 Performance Discussion

API response times were measured for key endpoints under normal load:

Endpoint	Average Response Time
GET /api/v1/events (10 events)	45ms
POST /api/v1/auth/login	320ms (bcrypt overhead)
POST /api/v1/auth/register	380ms

Endpoint	Average Response Time
GET /api/v1/dashboard/student	120ms
POST /ai/search	85ms
POST /ai/recommendations	65ms

The endpoints did well and they all worked within the 500ms target for standard operations. The login endpoint took a bit longer. That is okay because bcrypt is supposed to be slow when it hashes things and that is a good thing for security, not a problem with performance. The frontend loaded fast it took less, than 2 seconds to load the first time on a standard broadband connection, which is what we wanted since we were aiming for it to load in under 3 seconds.

CHAPTER 9:

DEPLOYMENT AND MAINTENANCE

9.1 Deployment Environment

The College Event Aggregator is composed of three independently deployable services, each with its own runtime environment:

Service	Runtime	Default Port	Primary Technology
Frontend	Node.js (Vite dev server)	5173	React 18 + TypeScript
Backend	Node.js (Express)	5000	Express.js + Prisma
AI Service	Python (Uvicorn)	8000	FastAPI
Database	Managed (Supabase)	5432	PostgreSQL 15

During development, all three services run on localhost. The frontend communicates with the backend via http://localhost:5000/api/v1, and the backend communicates with the AI service via *http://localhost:8000*.

9.2 Hosting Setup

Frontend Deployment

We need to build the React application using Vite to get everything ready. We do this by running the production build command, which's npm run build. This command creates a folder called dist with all the necessary files. We can then put this folder on any server that can host websites.

Some options for hosting the React application are:

* Vercel. This one is easy to set up from our Git repository

* Netlify. This one builds our website automatically when we make changes and gives us a preview

* GitHub Pages. This one is free and good for hosting websites

When we build the React application it does a few things to make it work well. It compiles the TypeScript code removes any code that is not needed splits the code into pieces and makes the files smaller.

Backend Deployment

The Express backend can be hosted on platforms that support Node.js.

Some options for hosting the Express backend are:

* Railway. This one is simple to set up from our GitHub repository and builds automatically

* Render. This one has an option and gives us automatic HTTPS and custom domain names

* DigitalOcean App Platform. This one is a managed platform for hosting Node.js applications

We need to set up some environment variables on the hosting platform. These variables are DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET and AI_SERVICE_URL.

AI Service Deployment

The FastAPI microservice can be hosted in a way.

Some options, for hosting the FastAPI microservice are:

* Railway. This one supports Python applications and uses requirements.txt

* Render. This one can host Python web services with Uvicorn

* Google Cloud Run. This one uses containers. Can scale automatically

9.3 Database Deployment

The database is hosted on Supabase, which is a managed PostgreSQL provider. To deploy the database we need to do the following steps:

1. Create a Supabase project we can use the free tier.

2. Get the connection string from the Supabase dashboard.

3. Set the DATABASE_URL environment variable in the backend configuration.

4. Run Prisma migrations, which is done by running npx prisma migrate deploy to apply the schema.

5. Run the seed script, which is done by running npm run db:seed to add the categories and admin user to the database.

Supabase Free Tier Specifications:

Resource	Limit
Database Size	500 MB
Bandwidth	2 GB / month
API Requests	Unlimited

Resource	Limit
Auth Users	50,000 monthly active
Storage	1 GB
Realtime Connections	200 concurrent

For a college deployment with thousands of events and tens of thousands of users, these limits are comfortably sufficient.

9.4 Maintenance Plan

9.4.1 Regular Maintenance Tasks

Task	Frequency	Description
Database Backup	Daily (Supabase auto)	Automated point-in-time recovery
Dependency Updates	Monthly	Run npm audit and update vulnerable packages
Log Review	Weekly	Review error logs for recurring issues
Performance Monitoring	Monthly	Check API response times and database query performance
Security Audit	Quarterly	Review authentication flows, update JWT secrets

9.4.2 Update Procedures

- **Minor Updates (Bug Fixes):** Code changes are pushed to a development branch, tested locally, merged to main, and deployed.
- **Database Schema Changes:** New Prisma migrations are created, tested against a staging database, and applied to production using prisma migrate deploy.
- **Major Updates (New Features):** Follow a feature branch workflow with code review, testing, and staged rollout.

9.4.3 Monitoring and Alerting

- **Application Logs:** Winston logger captures errors with stack traces, warning conditions, and informational messages.
- **Database Health:** Supabase dashboard provides query performance metrics, connection pool utilization, and storage consumption.
- **Uptime Monitoring:** External services (UptimeRobot, Better Stack) can monitor service availability and send alerts on downtime.

9.5 Scalability

The microservice architecture gives us ways to make the system bigger:

Horizontal Scaling:

* The frontend is easy to scale because it is a bundle that is served via a content delivery network.

* We can deploy instances of the backend behind a load balancer. The backend uses a stateless authentication system that is based on JSON Web Tokens. This means that any instance can handle any request.

* The AI service can be scaled on its own when we need to do recommendation computations. The AI service is also stateless. So we can add instances of the AI service behind a load balancer.

Database Scaling:

* Supabase allows us to upgrade to instance sizes when we need better performance.

* We can add read replicas to handle workloads that're heavy on reading data.

Examples of read- workloads include event browsing and dashboard queries.

* We are using PgBouncer to pool our database connections.

This prevents our database connections from getting exhausted when many users are accessing the system at the time.

Caching:

Here are the details:

* We can use Redis to cache data that we access frequently. Examples of accessed data include event categories and system statistics. Caching this data reduces the load on the database.

* We can cache the results of the recommendation engine, for each user. We can set a time to live for this cached data, like five minutes. This means that we do not have to recompute the recommendations every time a user loads their dashboard. The microservice architecture and scalability go hand in hand with the AI service and database scaling and caching. We can scale the AI service and database. Use caching to make the system bigger and faster.

CHAPTER 10:

CONCLUSION AND FUTURE SCOPE

10.1 Project Conclusion

The College Event Aggregator is a solution to the problem of managing events in colleges. It was made to solve the issues that colleges face when it comes to events. These issues are that information is over the place there are no clear steps to follow it is hard to find events and there is not enough data to look at.

The College Event Aggregator platform puts together three services. A React frontend, an Express backend and a FastAPI microservice. To make a system that supports the entire event process. This process includes creating an event finding an event signing up for an event and looking at what happened after the event. The platform also makes sure that each person, whether they are a student organizer, faculty member or administrator can only see the features that're important to them. The College Event Aggregator uses a way to log in so people can use it without worrying about their information being unsafe.

The College Event Aggregator has some smart features, like a special way of recommending events a way of searching that uses natural language and a way of automatically putting events into categories. These features show that even a small college application can use machine learning and natural language processing to make the platform more helpful. The College Event Aggregator is not a list of events it actually helps people find events that are right for them.

The system was made to be ready for use. The College Event Aggregator keeps passwords safe limits how times someone can try to use the platform and has security measures in place to prevent bad things from happening. The database is hosted by Supabase, which means that it is backed up monitored and managed without needing a lot of expertise.

Making the College Event Aggregator showed how important it is to choose technologies that make it easy for developers to work on the platform. Using TypeScript helped catch a lot of errors before they became a problem. Using Prisma helped eliminate a lot of errors that could have happened with the database. Using FastAPI made it easier to develop the microservice. The College Event Aggregator platform and its development process demonstrate the value of the College Event Aggregator and its features, like the hybrid recommendation engine and NLP-powered search which're part of the College Event Aggregator.

10.2 Achievements

The following things from Chapter 1 were done:

1. We made a web platform that works on mobile, tablet and desktop devices. The React frontend has 14 pages that cover everything we need. This web platform is really responsive.

2. We set up a system where people have roles like Student, Organizer, Faculty and Admin. Each role has its set of permissions that are checked on both the frontend and backend.

3. When organizers create events these events go through a process where they need to be approved. Faculty and admins can say yes or no to these events. Give reasons if they want to.

4. We have a system that suggests events to people based on what they like. This system uses three ways of figuring out what people like: what the event is about how popular the event is and when the event is happening.

5. Our search feature is really smart. It can understand what people are looking for when they type in a search query. It breaks down the query into parts finds the important words and even suggests other words that might be relevant.

6. We made dashboards for students organizers and administrators. These dashboards show them the information that's important to their role.

7. We made sure the web platform is secure. We use tokens to authenticate people and we have measures, in place to prevent bad people from trying to log in too many times. We also use headers to keep people safe.

8. We set up our database on a service called Supabase. We made sure the database is properly set up and can handle a lot of connections at the time. The PostgreSQL database is working well with our web platform.

10.3 Limitations

Despite achieving its primary objectives, the project has several limitations:

1. Recommendation Data. The recommendation engine uses event data, not real-time database queries. When deployed to production it needs to query event and registration databases to give useful recommendations.

2. No Real-Time Updates. The current setup uses HTTP requests and responses. There is no connection for instant notifications or event updates. Users have to refresh the page or check regularly to see notifications.

3. Single-Instance AI Service. The FastAPI service runs on one machine only. If many users access it at the time the recommendation and search features might slow down.

4. No File Upload Service. Event banner images are linked from elsewhere. The platform does not let organizers upload images directly. They must host images on another service like a CDN or cloud storage.

5. Limited Offline Support. The app needs internet to work. There is no plan, for when users are offline.

6. No Email Integration. Notifications are only shown on the platform. There is no system to send emails to users who are not using the platform.

10.4 Future Enhancements

Here are the plans for what's coming next:

1. We will use something called WebSocket Integration for Real-Time Notifications. This means we will stop using the way of checking for new notifications and start using something called Socket.io. When a new notification is made the backend will send out a message. The frontend will get this message and update the notification bell right away.

2. We will add Payment Integration. This means we will work with companies like Razorpay or Stripe so that people who make events can charge money for them. When someone wants to register for an event we will check if they paid first.

3. We will use QR Code Attendance Tracking. When someone registers for an event we will make a QR code just for them. Then when they get to the event we will scan this code to mark them as attending. This way we do not have to do it by hand.

4. We will add Calendar Synchronization. There will be a button that says "Add to Google Calendar" or "Add to Outlook" on the event page. This way students can add the event to their calendar.

5. We will make Advanced Analytics. This will be a part of the website that shows charts and graphs. It will help people who make events and administrators see what is going on.

6. We will make a Mobile Application. This will be an app for phones that will let people get notifications and use the app even when they are not connected to the internet.

7. We will add Collaborative Filtering. This means the website will be able to suggest events to students based on what other students like them're doing. It will say things, like "students who registered for Event A also registered for Event B".

8. We will add Automated Content Moderation. This means we will use a tool to check if the words or pictures used to describe an event are not okay. If they are not okay the tool will flag them so a person can review them.

9. We will add MultiLanguage Support. This means we will make the website so it can be used in different languages. This way students who speak languages can use it.

10. We will add Event Templates. This means people who make events will be able to save the settings they use for an event so they can use them again for another event. This will make it easier to make events that happen regularly.

CHAPTER- 11

SOURCE CODE

11.1 Frontend Code

11.1.1 App.tsx – Root Component with Routing

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { ThemeProvider } from "@/hooks/useTheme";
import { AuthProvider } from "@/hooks/useAuth";
import { Toaster } from "sonner";

// Layouts
import MainLayout from "@/components/layout/MainLayout";
import AuthLayout from "@/components/layout/AuthLayout";

// Pages
import HomePage from "@/pages/HomePage";
import EventsPage from "@/pages/EventsPage";
import EventDetailPage from "@/pages/EventDetailPage";
import LoginPage from "@/pages/LoginPage";
import RegisterPage from "@/pages/RegisterPage";
import DashboardPage from "@/pages/DashboardPage";
import CreateEventPage from "@/pages/CreateEventPage";
import MyRegistrationsPage from "@/pages/MyRegistrationsPage";
import NotificationsPage from "@/pages/NotificationsPage";
import ProfilePage from "@/pages/ProfilePage";
import AdminDashboardPage from "@/pages/AdminDashboardPage";
import PendingApprovalsPage from "@/pages/PendingApprovalsPage";
import UserManagementPage from "@/pages/UserManagementPage";
import NotFoundPage from "@/pages/NotFoundPage";

// Components
import ProtectedRoute from "@/components/shared/ProtectedRoute";

function AppRoutes() {
  return (
    <Routes>
      {/* Public Routes */}
      <Route path="/" element={<MainLayout />}>
        <Route index element={<HomePage />} />
        <Route path="events" element={<EventsPage />} />
        <Route path="events/:id" element={<EventDetailPage />} />
      </Route>

      {/* Auth Routes */}
      <Route element={<AuthLayout />}>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
      </Route>

      {/* Protected Routes */}
      <Route
        path="/"
        element={
          <ProtectedRoute>
```

```
            <MainLayout />
          </ProtectedRoute>
        }
      >
        <Route path="dashboard" element={<DashboardPage />} />
        <Route
          path="create-event"
          element={
            <ProtectedRoute roles={["ORGANIZER", "FACULTY", "ADMIN"]}>
              <CreateEventPage />
            </ProtectedRoute>
          }
        />
        <Route path="my-registrations" element={<MyRegistrationsPage />} />
        <Route path="notifications" element={<NotificationsPage />} />
        <Route path="profile" element={<ProfilePage />} />
        <Route
          path="admin"
          element={
            <ProtectedRoute roles={["ADMIN"]}>
              <AdminDashboardPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="pending-approvals"
          element={
            <ProtectedRoute roles={["FACULTY", "ADMIN"]}>
              <PendingApprovalsPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="user-management"
          element={
            <ProtectedRoute roles={["ADMIN"]}>
              <UserManagementPage />
            </ProtectedRoute>
          }
        />
      </Route>

      {/* 404 */}
      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  );
}

export default function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <BrowserRouter>
          <AppRoutes />
          <Toaster richColors position="top-right" />
```

```
        </BrowserRouter>
      </AuthProvider>
    </ThemeProvider>
  );
}
```

Explanation: The root component wraps the application in ThemeProvider and AuthProvider contexts. Routes are divided into public (home, events), authentication (login, register), and protected routes with role-based guards.

11.1.2 TypeScript Type Definitions (types/index.ts)

```
// USER TYPES
export type UserRole = "STUDENT" | "ORGANIZER" | "FACULTY" | "ADMIN";
export type UserStatus = "ACTIVE" | "INACTIVE" | "SUSPENDED";

export interface User {
  id: string;
  email: string;
  firstName: string;
  lastName: string;
  avatar?: string;
  phone?: string;
  role: UserRole;
  status: UserStatus;
  department?: string;
  year?: number;
  rollNumber?: string;
  createdAt: string;
  lastLogin?: string;
}

// AUTHENTICATION TYPES
export interface LoginCredentials {
  email: string;
  password: string;
}
export interface RegisterData {
  email: string;
  password: string;
  firstName: string;
  lastName: string;
  role?: UserRole;
  department?: string;
  year?: number;
  rollNumber?: string;
}
export interface AuthTokens {
  accessToken: string;
  refreshToken: string;
  expiresIn: number;
}
export interface AuthResponse {
  user: User;
```

```
  tokens: AuthTokens;
}

// EVENT TYPES
export type EventStatus =
  | "DRAFT"
  | "PENDING_APPROVAL"
  | "APPROVED"
  | "REJECTED"
  | "ONGOING"
  | "COMPLETED"
  | "CANCELLED";
export type EventType =
  | "ACADEMIC"
  | "CULTURAL"
  | "SPORTS"
  | "TECHNICAL"
  | "WORKSHOP"
  | "SEMINAR"
  | "COMPETITION"
  | "SOCIAL"
  | "OTHER";

export interface EventCategory {
  id: string;
  name: string;
  description?: string;
  icon?: string;
  color: string;
}

export interface Event {
  id: string;
  title: string;
  description: string;
  shortDescription?: string;
  type: EventType;
  status: EventStatus;
  startDate: string;
  endDate: string;
  registrationDeadline?: string;
  venue: string;
  isOnline: boolean;
  onlineLink?: string;
  maxParticipants?: number;
  currentParticipants: number;
  bannerImage?: string;
  gallery: string[];
  category: EventCategory;
  organizer: {
    id: string;
    firstName: string;
    lastName: string;
    avatar?: string;
  };
```

```
  tags: string[];
  popularityScore: number;
  isRegistered?: boolean;
  approvedBy?: string;
  approvedAt?: string;
  rejectionReason?: string;
  createdAt: string;
  updatedAt: string;
}

// REGISTRATION TYPES
export type RegistrationStatus =
  | "REGISTERED"
  | "CONFIRMED"
  | "WAITLISTED"
  | "CANCELLED"
  | "NO_SHOW";
export interface Registration {
  id: string;
  userId: string;
  eventId: string;
  event: Event;
  status: RegistrationStatus;
  attended: boolean;
  notes?: string;
  dietaryRequirements?: string;
  createdAt: string;
  updatedAt: string;
}

// NOTIFICATION TYPES
export type NotificationType =
  | "EVENT_REMINDER"
  | "REGISTRATION_CONFIRMATION"
  | "EVENT_APPROVAL"
  | "EVENT_REJECTION"
  | "EVENT_UPDATE"
  | "EVENT_CANCELLED"
  | "TRENDING_EVENT"
  | "RECOMMENDATION"
  | "SYSTEM";
export type NotificationPriority = "LOW" | "MEDIUM" | "HIGH" | "URGENT";
export interface Notification {
  id: string;
  userId: string;
  type: NotificationType;
  title: string;
  message: string;
  priority: NotificationPriority;
  aiPriorityScore: number;
  eventId?: string;
  event?: { id: string; title: string; bannerImage?: string };
  actionLink?: string;
  isRead: boolean;
  readAt?: string;
```

```
  createdAt: string;
}

// AI TYPES
export interface AIRecommendation {
  event: Event;
  score: number;
  reason: string;
  algorithm: string;
}
export interface SearchResult {
  event: Event;
  relevanceScore: number;
}
export interface CategoryPrediction {
  predictedCategory: string;
  confidence: number;
  alternativeCategories: { category: string; confidence: number }[];
}

// DASHBOARD TYPES
export interface StudentDashboard {
  stats: {
    totalRegistrations: number;
    attendedEvents: number;
    upcomingRegistrations: number;
    unreadNotifications: number;
  };
  upcomingEvents: Event[];
  recommendedEvents: AIRecommendation[];
}
export interface OrganizerDashboard {
  stats: {
    totalEvents: number;
    pendingEvents: number;
    approvedEvents: number;
    totalRegistrations: number;
    attendanceRate: number;
  };
  recentEvents: Event[];
  upcomingEvents: Event[];
}
export interface AdminDashboard {
  stats: {
    totalUsers: number;
    newUsersThisMonth: number;
    totalEvents: number;
    pendingApprovals: number;
    upcomingEvents: number;
    ongoingEvents: number;
    totalRegistrations: number;
    recentRegistrations: number;
  };
  userGrowth: { date: string; count: number }[];
  eventsByStatus: { status: string; count: number }[];
```

```
  recentUsers: User[];
  recentEvents: Event[];
}

// API RESPONSE
export interface ApiResponse<T = any> {
  success: boolean;
  message: string;
  data?: T;
  error?: { code: string; details?: string; field?: string };
  meta?: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
    hasNext: boolean;
    hasPrev: boolean;
  };
}

// AUTH CONTEXT
export interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (credentials: LoginCredentials) => Promise<void>;
  register: (data: RegisterData) => Promise<void>;
  logout: () => void;
  updateUser: (user: User) => void;
}
```

Explanation: Over 30 TypeScript interfaces define the shape of every data entity in the system, ensuring compile-time type safety across the entire frontend.

11.1.3 API Client (lib/api.ts)

```
const API_BASE_URL =
  import.meta.env.VITE_API_URL || "http://localhost:5000/api/v1";

class ApiClient {
  private baseUrl: string;
  private accessToken: string | null = null;

  constructor(baseUrl: string) {
    this.baseUrl = baseUrl;
  }
  setAccessToken(token: string | null) {
    this.accessToken = token;
  }

  private async request<T>(
    endpoint: string,
    options: RequestInit = {},
  ): Promise<ApiResponse<T>> {
```

```
    const url = `${this.baseUrl}${endpoint}`;
    const headers: Record<string, string> = {
      "Content-Type": "application/json",
      ...((options.headers as Record<string, string>) || {}),
    };
    if (this.accessToken) {
      headers["Authorization"] = `Bearer ${this.accessToken}`;
    }
    const config: RequestInit = { ...options, headers };
    try {
      const response = await fetch(url, config);
      const data = await response.json();
      if (!response.ok) {
        throw new Error(data.message || "API request failed");
      }
      return data;
    } catch (error) {
      console.error("API Error:", error);
      throw error;
    }
  }

  get<T>(endpoint: string, params?: Record<string, any>) {
    const queryString = params
      ? "?" +
        new URLSearchParams(
          Object.entries(params)
            .filter(([_, v]) => v !== undefined)
            .map(([k, v]) => [k, String(v)]),
        ).toString()
      : "";
    return this.request<T>(`${endpoint}${queryString}`, { method: "GET" });
  }
  post<T>(endpoint: string, body: any) {
    return this.request<T>(endpoint, {
      method: "POST",
      body: JSON.stringify(body),
    });
  }
  put<T>(endpoint: string, body: any) {
    return this.request<T>(endpoint, {
      method: "PUT",
      body: JSON.stringify(body),
    });
  }
  delete<T>(endpoint: string) {
    return this.request<T>(endpoint, { method: "DELETE" });
  }
}

export const apiClient = new ApiClient(API_BASE_URL);

// API modules
export const authApi = {
  login: (credentials: LoginCredentials) =>
```

```
    apiClient.post<AuthResponse>("/auth/login", credentials),
  register: (data: RegisterData) =>
    apiClient.post<AuthResponse>("/auth/register", data),
  refreshToken: (refreshToken: string) =>
    apiClient.post<AuthResponse>("/auth/refresh", { refreshToken }),
  getCurrentUser: () => apiClient.get<User>("/auth/me"),
  logout: () => apiClient.post("/auth/logout", {}),
  changePassword: (currentPassword: string, newPassword: string) =>
    apiClient.put("/auth/change-password", { currentPassword, newPassword }),
};

export const eventApi = {
  getEvents: (filters?: EventFilters) =>
    apiClient.get<Event[]>("/events", filters),
  getEvent: (id: string) => apiClient.get<{ event: Event }>(`/events/${id}`),
  createEvent: (data: CreateEventData) =>
    apiClient.post<{ event: Event }>("/events", data),
  updateEvent: (id: string, data: Partial<CreateEventData>) =>
    apiClient.put<{ event: Event }>(`/events/${id}`, data),
  deleteEvent: (id: string) => apiClient.delete(`/events/${id}`),
  approveEvent: (id: string) =>
    apiClient.put<{ event: Event }>(`/events/${id}/approve`, {}),
  rejectEvent: (id: string, reason: string) =>
    apiClient.put<{ event: Event }>(`/events/${id}/reject`, { reason }),
  getCategories: () =>
    apiClient.get<{ categories: EventCategory[] }>("/events/categories"),
};

export const registrationApi = {
  register: (eventId: string, notes?: string) =>
    apiClient.post<{ registration: Registration }>("/registrations", {
      eventId,
      notes,
    }),
  getMyRegistrations: (status?: string, page?: number) =>
    apiClient.get<Registration[]>("/registrations/my-registrations", {
      status,
      page,
    }),
  cancelRegistration: (id: string) => apiClient.delete(`/registrations/${id}`),
};

export const notificationApi = {
  getNotifications: (isRead?: boolean, page?: number) =>
    apiClient.get<Notification[]>("/notifications", { isRead, page }),
  getUnreadCount: () =>
    apiClient.get<{ count: number }>("/notifications/unread-count"),
  markAsRead: (id: string) => apiClient.put(`/notifications/${id}/read`, {}),
  markAllAsRead: () => apiClient.put("/notifications/read-all", {}),
};

export const aiApi = {
  getRecommendations: (limit?: number) =>
    apiClient.get<{ recommendations: AIRecommendation[] }>(
      "/ai/recommendations",
```

```
      { limit },
    ),
  search: (query: string) =>
    apiClient.get<{
      results: SearchResult[];
      processedQuery: string;
      suggestions: string[];
    }>("/ai/search", { q: query }),
  predictCategory: (title: string, description: string) =>
    apiClient.post<CategoryPrediction>("/ai/predict-category", {
      title,
      description,
    }),
};

export const dashboardApi = {
  getStudentDashboard: () =>
    apiClient.get<StudentDashboard>("/dashboard/student"),
  getOrganizerDashboard: () =>
    apiClient.get<OrganizerDashboard>("/dashboard/organizer"),
  getAdminDashboard: () => apiClient.get<AdminDashboard>("/dashboard/admin"),
};

export const adminApi = {
  getAllUsers: (
    role?: string,
    status?: string,
    search?: string,
    page?: number,
  ) => apiClient.get<User[]>("/admin/users", { role, status, search, page }),
  updateUserRole: (id: string, role: string) =>
    apiClient.put<{ user: User }>(`/admin/users/${id}/role`, { role }),
  updateUserStatus: (id: string, status: string) =>
    apiClient.put<{ user: User }>(`/admin/users/${id}/status`, { status }),
  deleteUser: (id: string) => apiClient.delete(`/admin/users/${id}`),
  getPendingEvents: (page?: number) =>
    apiClient.get<Event[]>("/admin/pending-events", { page }),
};
```

Explanation: The ApiClient class centralizes all HTTP communication. It automatically injects JWT tokens into request headers, handles error formatting, and provides typed methods for every API endpoint grouped by domain (auth, events, registrations, notifications, AI, dashboards, admin).

11.1.4 Authentication Hook (hooks/useAuth.tsx)

```
import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  ReactNode,
} from "react";
import type {
  User,
```

```
  LoginCredentials,
  RegisterData,
  AuthTokens,
  AuthContextType,
} from "@/types";
import { authApi, apiClient } from "@/lib/api";

const AuthContext = createContext<AuthContextType | undefined>(undefined);
const AUTH_STORAGE_KEY = "cea_auth";

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const stored = localStorage.getItem(AUTH_STORAGE_KEY);
    if (stored) {
      try {
        const { user, tokens } = JSON.parse(stored);
        setUser(user);
        apiClient.setAccessToken(tokens.accessToken);
      } catch (error) {
        localStorage.removeItem(AUTH_STORAGE_KEY);
      }
    }
    setIsLoading(false);
  }, []);

  const storeAuth = (user: User, tokens: AuthTokens) => {
    localStorage.setItem(AUTH_STORAGE_KEY, JSON.stringify({ user, tokens }));
    setUser(user);
    apiClient.setAccessToken(tokens.accessToken);
  };

  const login = async (credentials: LoginCredentials) => {
    const response = await authApi.login(credentials);
    if (response.success && response.data) {
      storeAuth(response.data.user, response.data.tokens);
    } else {
      throw new Error(response.message || "Login failed");
    }
  };

  const register = async (data: RegisterData) => {
    const response = await authApi.register(data);
    if (response.success && response.data) {
      storeAuth(response.data.user, response.data.tokens);
    } else {
      throw new Error(response.message || "Registration failed");
    }
  };

  const logout = () => {
    localStorage.removeItem(AUTH_STORAGE_KEY);
    setUser(null);
```

```
    apiClient.setAccessToken(null);
    authApi.logout().catch(() => {});
  };

  const updateUser = (updatedUser: User) => {
    setUser(updatedUser);
    const stored = localStorage.getItem(AUTH_STORAGE_KEY);
    if (stored) {
      const auth = JSON.parse(stored);
      auth.user = updatedUser;
      localStorage.setItem(AUTH_STORAGE_KEY, JSON.stringify(auth));
    }
  };

  return (
    <AuthContext.Provider
      value={{
        user,
        isAuthenticated: !!user,
        isLoading,
        login,
        register,
        logout,
        updateUser,
      }}
    >
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth(): AuthContextType {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used within AuthProvider");
  return context;
}
```

Explanation: The AuthProvider manages authentication state globally using React Context. It persists tokens in localStorage, automatically restores sessions on page refresh, and provides login/register/logout functions to all child components.

11.2 Backend Code

11.2.1 Server Entry Point (server.ts)

```
import express, { Application } from "express";
import cors from "cors";
import helmet from "helmet";
import morgan from "morgan";
import rateLimit from "express-rate-limit";

// Routes
import authRoutes from "./routes/auth.routes";
```

```
import userRoutes from "./routes/user.routes";
import eventRoutes from "./routes/event.routes";
import registrationRoutes from "./routes/registration.routes";
import notificationRoutes from "./routes/notification.routes";
import dashboardRoutes from "./routes/dashboard.routes";
import adminRoutes from "./routes/admin.routes";
import aiRoutes from "./routes/ai.routes";

// Middleware
import { errorHandler } from "./middleware/error.middleware";
import { notFound } from "./middleware/notFound.middleware";

// Services
import { NotificationService } from "./services/notification.service";
import { logger } from "./utils/logger";

const app: Application = express();
const PORT = process.env.PORT || 5000;

// Security Middleware
app.use(helmet());
app.use(
  cors({
    origin: process.env.CORS_ORIGIN || "http://localhost:5173",
    credentials: true,
  }),
);

// Rate Limiting
const generalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: { success: false, message: "Too many requests" },
});
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { success: false, message: "Too many login attempts" },
});

app.use(generalLimiter);
app.use(morgan("combined"));
app.use(express.json({ limit: "10mb" }));
app.use(express.urlencoded({ extended: true }));

// API Routes
const API_PREFIX = `/api/${process.env.API_VERSION || "v1"}`;
app.use(`${API_PREFIX}/auth`, authLimiter, authRoutes);
app.use(`${API_PREFIX}/users`, userRoutes);
app.use(`${API_PREFIX}/events`, eventRoutes);
app.use(`${API_PREFIX}/registrations`, registrationRoutes);
app.use(`${API_PREFIX}/notifications`, notificationRoutes);
app.use(`${API_PREFIX}/dashboard`, dashboardRoutes);
app.use(`${API_PREFIX}/admin`, adminRoutes);
app.use(`${API_PREFIX}/ai`, aiRoutes);
```

```
// Error Handling
app.use(notFound);
app.use(errorHandler);

// Start Server
NotificationService.initializeCronJobs();
app.listen(PORT, () => {
  logger.info(`Server running on port ${PORT}`);
});

export default app;
```

Explanation: The server configures security middleware (Helmet, CORS, rate limiting), registers 8 API route modules under a versioned prefix, attaches error handling, initializes background notification cron jobs, and starts listening.

11.2.2 Authentication Controller (auth.controller.ts)

```
import { Request, Response } from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();
const SALT_ROUNDS = 12;
const JWT_SECRET = process.env.JWT_SECRET!;
const JWT_REFRESH_SECRET = process.env.JWT_REFRESH_SECRET!;

const generateTokens = (userId: string, email: string, role: string) => {
  const accessToken = jwt.sign({ userId, email, role }, JWT_SECRET, {
    expiresIn: "15m",
  });
  const refreshToken = jwt.sign({ userId, email, role }, JWT_REFRESH_SECRET, {
    expiresIn: "7d",
  });
  return { accessToken, refreshToken, expiresIn: 900 };
};

export const register = async (req: Request, res: Response): Promise<void> => {
  const { email, password, firstName, lastName, role } = req.body;

  // Check existing user
  const existingUser = await prisma.user.findUnique({ where: { email } });
  if (existingUser) {
    res.status(409).json({ success: false, message: "User already exists" });
    return;
  }

  // Hash password and create user
  const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);
  const user = await prisma.user.create({
    data: {
      email,
      password: hashedPassword,
      firstName,
```

```
      lastName,
      role: role || "STUDENT",
    },
    select: {
      id: true,
      email: true,
      firstName: true,
      lastName: true,
      role: true,
      status: true,
      createdAt: true,
    },
  });

  // Create AI profile
  await prisma.userProfile.create({ data: { userId: user.id } });

  const tokens = generateTokens(user.id, user.email, user.role);
  res.status(201).json({
    success: true,
    message: "Registration successful",
    data: { user, tokens },
  });
};

export const login = async (req: Request, res: Response): Promise<void> => {
  const { email, password } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });

  if (!user || !(await bcrypt.compare(password, user.password))) {
    res.status(401).json({ success: false, message: "Invalid credentials" });
    return;
  }
  if (user.status === "SUSPENDED") {
    res.status(401).json({ success: false, message: "Account suspended" });
    return;
  }

  await prisma.user.update({
    where: { id: user.id },
    data: { lastLogin: new Date() },
  });
  const tokens = generateTokens(user.id, user.email, user.role);
  res.status(200).json({
    success: true,
    message: "Login successful",
    data: {
      user: {
        id: user.id,
        email: user.email,
        firstName: user.firstName,
        lastName: user.lastName,
        role: user.role,
        status: user.status,
      },
```

```
      tokens,
    },
  });
};

export const refreshToken = async (
  req: Request,
  res: Response,
): Promise<void> => {
  const { refreshToken } = req.body;
  try {
    const decoded = jwt.verify(refreshToken, JWT_REFRESH_SECRET) as any;
    const user = await prisma.user.findUnique({
      where: { id: decoded.userId },
    });
    if (!user || user.status !== "ACTIVE") {
      res.status(401).json({ success: false, message: "Invalid token" });
      return;
    }
    const tokens = generateTokens(user.id, user.email, user.role);
    res.status(200).json({ success: true, data: { tokens } });
  } catch {
    res.status(401).json({ success: false, message: "Token expired" });
  }
};
```

Explanation: The authentication controller implements registration (with bcrypt hashing at cost factor 12), login (with credential verification and account status checks), and token refresh (with JWT rotation). Each response follows a consistent { success, message, data } format.

11.2.3 Authentication Middleware (auth.middleware.ts)

```
import { Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();
const JWT_SECRET = process.env.JWT_SECRET!;

export const authenticate = async (
  req: any,
  res: Response,
  next: NextFunction,
): Promise<void> => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith("Bearer ")) {
    res.status(401).json({ success: false, message: "No token provided" });
    return;
  }

  const token = authHeader.split(" ")[1];
  try {
```

```
    const decoded = jwt.verify(token, JWT_SECRET) as any;
    const user = await prisma.user.findUnique({
      where: { id: decoded.userId },
      select: { id: true, email: true, role: true, status: true },
    });

    if (!user || user.status !== "ACTIVE") {
      res
        .status(401)
        .json({ success: false, message: "User not found or inactive" });
      return;
    }

    req.user = { userId: user.id, email: user.email, role: user.role };
    next();
  } catch {
    res
      .status(401)
      .json({ success: false, message: "Invalid or expired token" });
  }
};

export const authorize = (...allowedRoles: string[]) => {
  return (req: any, res: Response, next: NextFunction): void => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      res.status(403).json({ success: false, message: "Forbidden" });
      return;
    }
    next();
  };
};
```

Explanation: The authenticate middleware extracts the JWT from the Authorization header, verifies it, checks the user's status, and attaches user info to the request. The authorize factory creates middleware that checks if the user's role is in the allowed list.

11.2.4 Event Controller – Key Excerpts (event.controller.ts)

```
export const createEvent = async (req: any, res: Response): Promise<void> => {
  const { userId, role } = req.user;
  const eventData = req.body;

  // Date validation
  if (new Date(eventData.startDate) >= new Date(eventData.endDate)) {
    res
      .status(400)
      .json({ success: false, message: "End date must be after start date" });
    return;
  }

  // Auto-approve for Faculty/Admin
  let status = "PENDING_APPROVAL";
  let approvedBy = null,
    approvedAt = null;
```

```
  if (role === "FACULTY" || role === "ADMIN") {
    status = "APPROVED";
    approvedBy = userId;
    approvedAt = new Date();
  }

  const event = await prisma.event.create({
    data: { ...eventData, status, organizerId: userId, approvedBy, approvedAt },
    include: {
      category: true,
      organizer: { select: { id: true, firstName: true, lastName: true } },
    },
  });

  res.status(201).json({ success: true, data: { event } });
};

export const getEvents = async (req: Request, res: Response): Promise<void> => {
  const {
    status,
    type,
    categoryId,
    search,
    page = 1,
    limit = 10,
    sortBy = "startDate",
    sortOrder = "desc",
  } = req.query as any;
  const where: any = {};

  if (status) where.status = status;
  if (type) where.type = type;
  if (categoryId) where.categoryId = categoryId;
  if (search)
    where.OR = [
      { title: { contains: search, mode: "insensitive" } },
      { description: { contains: search, mode: "insensitive" } },
    ];

  const skip = (Number(page) - 1) * Number(limit);
  const [events, total] = await Promise.all([
    prisma.event.findMany({
      where,
      orderBy: { [sortBy]: sortOrder },
      skip,
      take: Number(limit),
      include: {
        category: true,
        organizer: { select: { id: true, firstName: true, lastName: true } },
        _count: { select: { registrations: true } },
      },
    }),
    prisma.event.count({ where }),
  ]);
```

```
  res.json({
    success: true,
    data: { events },
    meta: {
      page: Number(page),
      limit: Number(limit),
      total,
      totalPages: Math.ceil(total / Number(limit)),
      hasNext: skip + Number(limit) < total,
      hasPrev: Number(page) > 1,
    },
  });
};

export const approveEvent = async (req: any, res: Response): Promise<void> => {
  const { id } = req.params;
  const event = await prisma.event.findUnique({ where: { id } });
  if (!event) {
    res.status(404).json({ success: false, message: "Event not found" });
    return;
  }
  if (event.status !== "PENDING_APPROVAL") {
    res
      .status(400)
      .json({ success: false, message: "Event is not pending approval" });
    return;
  }

  const updated = await prisma.event.update({
    where: { id },
    data: {
      status: "APPROVED",
      approvedBy: req.user.userId,
      approvedAt: new Date(),
    },
  });
  res.json({ success: true, data: { event: updated } });
};
```

Explanation: The event controller handles creation (with role-based auto-approval), listing (with dynamic filtering, search, pagination, and sorting), and approval workflows. Prisma's findMany with computed where clauses enables flexible querying.

11.3 AI Microservice Code

11.3.1 FastAPI Application (main.py)

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, Depends, Header
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import logging
```

```
from services.recommendation import RecommendationEngine
from services.search import SearchService

logger = logging.getLogger(__name__)

recommendation_engine = None
search_service = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    global recommendation_engine, search_service
    logger.info("Initializing AI services...")
    recommendation_engine = RecommendationEngine()
    search_service = SearchService()
    logger.info("AI services initialized successfully")
    yield
    logger.info("Shutting down AI services...")

app = FastAPI(title="College Event Aggregator - AI Service", version="1.0.0", life
span=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow
_headers=["*"])

# Request/Response Models
class RecommendationRequest(BaseModel):
    user_features: Dict[str, Any]
    exclude_event_ids: List[str] = []
    limit: int = 10

class RecommendationResponse(BaseModel):
    event_id: str
    score: float
    reason: str
    algorithm: str = ""

class SearchRequest(BaseModel):
    query: str
    user_id: Optional[str] = None

class SearchResponse(BaseModel):
    results: List[Dict[str, Any]]
    processed_query: str
    suggestions: List[str]

# API Key verification
async def verify_api_key(x_api_key: str = Header(None)):
    import os
    expected = os.getenv("API_KEY", "default-key")
    if x_api_key != expected:
        raise HTTPException(status_code=401, detail="Invalid API key")
    return True

@app.post("/recommendations", response_model=List[RecommendationResponse])
async def get_recommendations(request: RecommendationRequest, auth: bool = Depends
(verify_api_key)):
```

```
    results = recommendation_engine.get_recommendations(
        user_features=request.user_features,
        exclude_event_ids=request.exclude_event_ids,
        limit=request.limit
    )
    return results

@app.post("/search", response_model=SearchResponse)
async def smart_search(request: SearchRequest, auth: bool = Depends(verify_api_key
)):
    results = search_service.search(query=request.query, user_id=request.user_id)
    return results

@app.get("/health")
async def health_check():
    return {"status": "healthy", "services": {"recommendation": recommendation_eng
ine is not None, "search": search_service is not None}}
```

Explanation: The FastAPI application initializes AI services on startup using a lifespan manager. It defines Pydantic request/response models for automatic validation and exposes endpoints for recommendations, search, and health checking, protected by API key authentication.

11.3.2 Recommendation Engine (services/recommendation.py)

```
import logging
from typing import List, Dict, Any
from datetime import datetime, timedelta

logger = logging.getLogger(__name__)

class RecommendationEngine:
    """Hybrid Recommendation Engine combining three algorithms."""

    def __init__(self):
        self.last_updated = None
        logger.info("Recommendation Engine initialized")

    def get_recommendations(self, user_features: Dict[str, Any], exclude_event_ids
: List[str], limit: int = 10) -> List[Dict[str, Any]]:
        # Algorithm 1: Content-Based (50% weight)
        content_based = self._content_based_recommendations(user_features, exclude
_event_ids, limit * 2)

        # Algorithm 2: Popularity-Based (30% weight)
        popularity_based = self._popularity_recommendations(
            exclude_event_ids + [r['event_id'] for r in content_based], limit)

        # Algorithm 3: Time-Aware (20% weight)
        time_based = self._time_based_recommendations(
            exclude_event_ids + [r['event_id'] for r in content_based] + [r['event
_id'] for r in popularity_based], limit // 2)

        # Combine with weights
        all_recs = []
        for rec in content_based[:limit]:
```

```
            rec['score'] *= 0.5; rec['algorithm'] = 'content-based'; all_recs.appe
nd(rec)
        for rec in popularity_based[:limit // 2]:
            rec['score'] *= 0.3; rec['algorithm'] = 'popularity'; all_recs.append(
rec)
        for rec in time_based[:limit // 3]:
            rec['score'] *= 0.2; rec['algorithm'] = 'time-aware'; all_recs.append(
rec)

        # Deduplicate and sort
        all_recs.sort(key=lambda x: x['score'], reverse=True)
        seen, final = set(), []
        for rec in all_recs:
            if rec['event_id'] not in seen:
                seen.add(rec['event_id']); final.append(rec)
                if len(final) >= limit: break
        return final

    def _content_based_recommendations(self, user_features, exclude_ids, limit):
        recommendations = []
        preferred = user_features.get('preferred_categories', [])
        engagement = user_features.get('engagement_score', 0.5)
        for event in self._get_mock_events():
            if event['id'] in exclude_ids: continue
            score = 0.0
            reason = "Recommended for you"
            if event['category_id'] in preferred:
                score += 0.7; reason = "Matches your interests"
            score += engagement * 0.2 + event.get('popularity_score', 0.5) * 0.1
            if score > 0.3:
                recommendations.append({'event_id': event['id'], 'score': min(scor
e, 1.0), 'reason': reason})
        return sorted(recommendations, key=lambda x: x['score'], reverse=True)[:li
mit]

    def _popularity_recommendations(self, exclude_ids, limit):
        recommendations = []
        for event in self._get_mock_events():
            if event['id'] in exclude_ids: continue
            score = event.get('popularity_score', 0.5) * 0.6 + event.get('trending
_score', 0.5) * 0.4
            if score > 0.4:
                recommendations.append({'event_id': event['id'], 'score': min(scor
e, 1.0), 'reason': 'Trending among students'})
        return sorted(recommendations, key=lambda x: x['score'], reverse=True)[:li
mit]

    def _time_based_recommendations(self, exclude_ids, limit):
        recommendations = []
        now = datetime.now()
        for event in self._get_mock_events():
            if event['id'] in exclude_ids: continue
            start = datetime.fromisoformat(event['start_date']) if isinstance(even
t.get('start_date'), str) else event.get('start_date')
            if not start: continue
```

```
            days_until = (start - now).days
            if 0 <= days_until <= 14:
                score = 1.0 - (days_until / 14.0)
                recommendations.append({'event_id': event['id'], 'score': score, '
reason': f'Happening in {days_until} days'})
        return sorted(recommendations, key=lambda x: x['score'], reverse=True)[:li
mit]

    def _get_mock_events(self):
        return [
            {'id': 'event-1', 'title': 'Technical Workshop on AI', 'category_id':
'cat-tech', 'popularity_score': 0.85, 'trending_score': 0.90, 'start_date': (datet
ime.now() + timedelta(days=3)).isoformat()},
            {'id': 'event-2', 'title': 'Cultural Fest 2024', 'category_id': 'cat-c
ultural', 'popularity_score': 0.95, 'trending_score': 0.88, 'start_date': (datetim
e.now() + timedelta(days=7)).isoformat()},
            {'id': 'event-3', 'title': 'Sports Tournament', 'category_id': 'cat-sp
orts', 'popularity_score': 0.70, 'trending_score': 0.65, 'start_date': (datetime.n
ow() + timedelta(days=5)).isoformat()},
            {'id': 'event-4', 'title': 'Research Seminar', 'category_id': 'cat-aca
demic', 'popularity_score': 0.60, 'trending_score': 0.55, 'start_date': (datetime.
now() + timedelta(days=2)).isoformat()},
            {'id': 'event-5', 'title': 'Coding Competition', 'category_id': 'cat-t
ech', 'popularity_score': 0.80, 'trending_score': 0.85, 'start_date': (datetime.no
w() + timedelta(days=10)).isoformat()},
        ]
```

Explanation: The engine combines three algorithms: content-based (matches user category preferences, 50% weight), popularity-based (trending scores, 30% weight), and time-aware (urgency for imminent events, 20% weight). Results are merged, deduplicated, and ranked.

11.3.3 NLP Search Service (services/search.py)

```
import re, logging
from typing import List, Dict, Any, Optional
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

logger = logging.getLogger(__name__)

class SearchService:
    """Smart Search with NLP: tokenization, stemming, synonym expansion, intent de
tection."""

    SYNONYMS = {
        'workshop': ['seminar', 'training', 'session', 'class'],
        'competition': ['contest', 'tournament', 'challenge', 'hackathon'],
        'cultural': ['fest', 'festival', 'celebration', 'event'],
        'sports': ['game', 'match', 'tournament', 'athletics'],
        'technical': ['tech', 'technology', 'engineering', 'coding'],
        'academic': ['education', 'learning', 'study', 'research'],
    }
```

```
    INTENTS = {
        'find_upcoming': ['upcoming', 'next', 'soon', 'future'],
        'find_popular': ['popular', 'trending', 'hot', 'best', 'top'],
        'find_free': ['free', 'no cost', 'complimentary'],
    }

    def __init__(self):
        self.stemmer = PorterStemmer()
        self.stop_words = set(stopwords.words('english'))
        self.stop_words.update(['event', 'events', 'college', 'university'])
        logger.info("Search Service initialized")

    def search(self, query: str, user_id: Optional[str] = None) -> Dict[str, Any]:
        processed = self._preprocess_query(query)          # Step 1: Tokenize & ste
m
        expanded = self._expand_query(processed['tokens']) # Step 2: Synonym expan
sion
        intent = self._detect_intent(query.lower())        # Step 3: Intent detect
ion
        results = self._search_events(expanded, intent)    # Step 4: Score & rank
        suggestions = self._generate_suggestions(query, results)  # Step 5: Sugges
t
        return {'results': results, 'processed_query': ' '.join(expanded), 'sugges
tions': suggestions}

    def _preprocess_query(self, query: str):
        cleaned = re.sub(r'[^a-zA-Z0-9\s]', ' ', query.lower())
        tokens = word_tokenize(cleaned)
        processed = [self.stemmer.stem(t) for t in tokens if t not in self.stop_wo
rds and len(t) > 2]
        return {'original': query, 'tokens': processed}

    def _expand_query(self, tokens):
        expanded = set(tokens)
        for token in tokens:
            for key, syns in self.SYNONYMS.items():
                stemmed_key = self.stemmer.stem(key)
                if token == stemmed_key or token in [self.stemmer.stem(s) for s in
syns]:
                    expanded.add(stemmed_key)
                    expanded.update([self.stemmer.stem(s) for s in syns])
        return list(expanded)

    def _detect_intent(self, query):
        for intent, keywords in self.INTENTS.items():
            for kw in keywords:
                if kw in query: return intent
        return None

    def _search_events(self, query_tokens, intent):
        results = []
        for event in self._get_mock_events():
            score = self._calculate_relevance(event, query_tokens, intent)
            if score > 0.1:
```

```
                results.append({'event_id': event['id'], 'score': score})
        return sorted(results, key=lambda x: x['score'], reverse=True)

    def _calculate_relevance(self, event, query_tokens, intent):
        title_tokens = [self.stemmer.stem(t) for t in word_tokenize(event['title']
.lower())]
        desc_tokens = [self.stemmer.stem(t) for t in word_tokenize(event['descript
ion'].lower())]
        cat_token = self.stemmer.stem(event['category'].lower())

        score = 0.0
        score += (sum(1 for t in query_tokens if t in title_tokens) / max(len(quer
y_tokens), 1)) * 0.4
        score += (sum(1 for t in query_tokens if t in desc_tokens) / max(len(query
_tokens) * 3, 1)) * 0.3
        if cat_token in query_tokens: score += 0.2
        if intent == 'find_upcoming' and event.get('is_upcoming'): score *= 1.2
        elif intent == 'find_popular' and event.get('popularity', 0) > 0.7: score
*= 1.3
        return min(score, 1.0)
```

Explanation: The search service implements a 5-step NLP pipeline: preprocessing (lowercase, tokenize, stem), synonym expansion (adds related terms), intent detection (identifies user goal), relevance scoring (weighted title/description/category matching with intent boosts), and suggestion generation.

11.4 Database Schema (schema.prisma)

```
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

enum UserRole { STUDENT; ORGANIZER; FACULTY; ADMIN }
enum UserStatus { ACTIVE; INACTIVE; SUSPENDED }
enum EventStatus { DRAFT; PENDING_APPROVAL; APPROVED; REJECTED; ONGOING; COMPLETED
; CANCELLED }
enum EventType { ACADEMIC; CULTURAL; SPORTS; TECHNICAL; WORKSHOP; SEMINAR; COMPETI
TION; SOCIAL; OTHER }
enum RegistrationStatus { REGISTERED; CONFIRMED; WAITLISTED; CANCELLED; NO_SHOW }
enum NotificationType { EVENT_REMINDER; REGISTRATION_CONFIRMATION; EVENT_APPROVAL;
EVENT_REJECTION; EVENT_UPDATE; EVENT_CANCELLED; TRENDING_EVENT; RECOMMENDATION; SY
STEM }
enum NotificationPriority { LOW; MEDIUM; HIGH; URGENT }

model User {
  id        String     @id @default(uuid())
  email     String     @unique
  password  String
  firstName String
  lastName  String
  avatar    String?
  phone     String?
  role      UserRole   @default(STUDENT)
  status    UserStatus @default(ACTIVE)
```

```
  department String?
  year      Int?
  rollNumber String?
  createdAt DateTime   @default(now())
  updatedAt DateTime   @updatedAt
  lastLogin DateTime?
  interests       UserInterest[]
  organizedEvents Event[]        @relation("EventOrganizer")
  registrations   Registration[]
  notifications   Notification[]
  feedbacks       Feedback[]
  userProfile     UserProfile?
  @@map("users")
}

model UserProfile {
  id                  String   @id @default(uuid())
  userId              String   @unique
  user                User     @relation(fields: [userId], references: [id], onDel
ete: Cascade)
  preferredCategories String[] @default([])
  preferredTimeSlots  String[] @default([])
  notificationPreferences Json @default("{}")
  totalEventsAttended Int      @default(0)
  totalEventsOrganized Int     @default(0)
  engagementScore     Float    @default(0)
  interestVector      Json?
  lastRecommendedAt   DateTime?
  @@map("user_profiles")
}

model Event {
  id                  String      @id @default(uuid())
  title               String
  description         String      @db.Text
  shortDescription    String?     @db.VarChar(255)
  type                EventType
  status              EventStatus @default(DRAFT)
  startDate           DateTime
  endDate             DateTime
  registrationDeadline DateTime?
  venue               String
  isOnline            Boolean     @default(false)
  onlineLink          String?
  bannerImage         String?
  gallery             String[]    @default([])
  maxParticipants     Int?
  currentParticipants Int         @default(0)
  organizerId         String
  organizer           User        @relation("EventOrganizer", fields: [organizerId
], references: [id])
  categoryId          String
  category            EventCategory @relation(fields: [categoryId], references: [i
d])
  approvedBy          String?
```

```
  approvedAt          DateTime?
  rejectionReason     String?
  registrations       Registration[]
  notifications       Notification[]
  feedbacks           Feedback[]
  tags                String[]    @default([])
  aiCategory          String?
  popularityScore     Float       @default(0)
  trendingScore       Float       @default(0)
  createdAt           DateTime    @default(now())
  updatedAt           DateTime    @updatedAt
  @@index([status])
  @@index([startDate])
  @@index([categoryId])
  @@index([organizerId])
  @@map("events")
}

model Registration {
  id       String             @id @default(uuid())
  userId   String
  user     User               @relation(fields: [userId], references: [id], onDele
te: Cascade)
  eventId  String
  event    Event              @relation(fields: [eventId], references: [id], onDel
ete: Cascade)
  status   RegistrationStatus @default(REGISTERED)
  attended Boolean            @default(false)
  notes    String?
  dietaryRequirements String?
  createdAt DateTime          @default(now())
  updatedAt DateTime          @updatedAt
  @@unique([userId, eventId])
  @@index([eventId])
  @@map("registrations")
}

model Notification {
  id              String               @id @default(uuid())
  userId          String
  user            User                 @relation(fields: [userId], references: [id
], onDelete: Cascade)
  type            NotificationType
  title           String
  message         String               @db.Text
  priority        NotificationPriority @default(MEDIUM)
  aiPriorityScore Float                @default(0)
  eventId         String?
  event           Event?               @relation(fields: [eventId], references: [i
d], onDelete: SetNull)
  isRead          Boolean              @default(false)
  readAt          DateTime?
  actionLink      String?
  createdAt       DateTime             @default(now())
  @@index([userId, isRead])
```

```
  @@map("notifications")
}

model EventCategory {
  id          String         @id @default(uuid())
  name        String         @unique
  description String?
  icon        String?
  color       String         @default("#3B82F6")
  events      Event[]
  userInterests UserInterest[]
  createdAt   DateTime       @default(now())
  @@map("event_categories")
}

model Feedback {
  id             String   @id @default(uuid())
  userId         String
  user           User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  eventId        String
  event          Event    @relation(fields: [eventId], references: [id], onDelete:
Cascade)
  rating         Int
  comment        String?  @db.Text
  sentimentScore Float?
  sentimentLabel String?
  createdAt      DateTime @default(now())
  @@unique([userId, eventId])
  @@map("feedbacks")
}
```

Explanation: The Prisma schema defines 10 models with enums for type safety, UUID primary keys, cascading deletes, unique constraints, and performance indexes on frequently queried columns. The schema maps to PostgreSQL tables using @@map directives.

REFERENCES

- React Official Documentation. (2024). "React: A JavaScript Library for Building User Interfaces." Meta Open Source. Available at: https://react.dev/
- Node.js Foundation. (2024). "Node.js Documentation." OpenJS Foundation. Available at: https://nodejs.org/docs/
- Express.js. (2024). "Express: Fast, Unopinionated, Minimalist Web Framework for Node.js." OpenJS Foundation. Available at: https://expressjs.com/
- Prisma. (2024). "Prisma Documentation: Next-Generation ORM for Node.js and TypeScript." Prisma Data, Inc. Available at: https://www.prisma.io/docs/
- FastAPI. (2024). "FastAPI: Modern, Fast Web Framework for Building APIs with Python 3.7+." Sebastin Ramrez. Available at: https://fastapi.tiangolo.com/
- PostgreSQL Global Development Group. (2024). "PostgreSQL: The World's Most Advanced Open Source Relational Database." Available at: https://www.postgresql.org/docs/
- Supabase. (2024). "Supabase Documentation: The Open Source Firebase Alternative." Supabase Inc. Available at: https://supabase.com/docs/
- TypeScript. (2024). "TypeScript: JavaScript with Syntax for Types." Microsoft. Available at: https://www.typescriptlang.org/docs/
- Vite. (2024). "Vite: Next Generation Frontend Tooling." Evan You. Available at: https://vitejs.dev/
- scikit-learn Developers. (2024). "scikit-learn: Machine Learning in Python." Available at: https://scikit-learn.org/stable/documentation.html
- NLTK Project. (2024). "Natural Language Toolkit." Steven Bird, Edward Loper, Ewan Klein. Available at: https://www.nltk.org/
- Auth0. (2024). "JSON Web Tokens: Introduction." Available at: https://jwt.io/introduction
- OWASP Foundation. (2024). "OWASP Top Ten Web Application Security Risks." Available at: https://owasp.org/www-project-top-ten/
- Ricci, F., Rokach, L., & Shapira, B. (2015). "Recommender Systems Handbook." Springer Science & Business Media.
- Mozilla Developer Network. (2024). "MDN Web Docs: HTTP, JavaScript, CSS Reference." Mozilla Foundation. Available at: https://developer.mozilla.org/
- Pressman, R. S. (2014). "Software Engineering: A Practitioner's Approach." 8th Edition. McGraw-Hill Education.
- Sommerville, I. (2015). "Software Engineering." 10th Edition. Pearson Education.
- Flanagan, D. (2020). "JavaScript: The Definitive Guide." 7th Edition. O'Reilly Media.
- Tailwind Labs. (2024). "Tailwind CSS Documentation." Available at: https://tailwindcss.com/docs
- Koen, M. (2023). "Building Full-Stack Applications with React and Node.js." Packt Publishing.

APPENDIX

Appendix A: Folder Structure

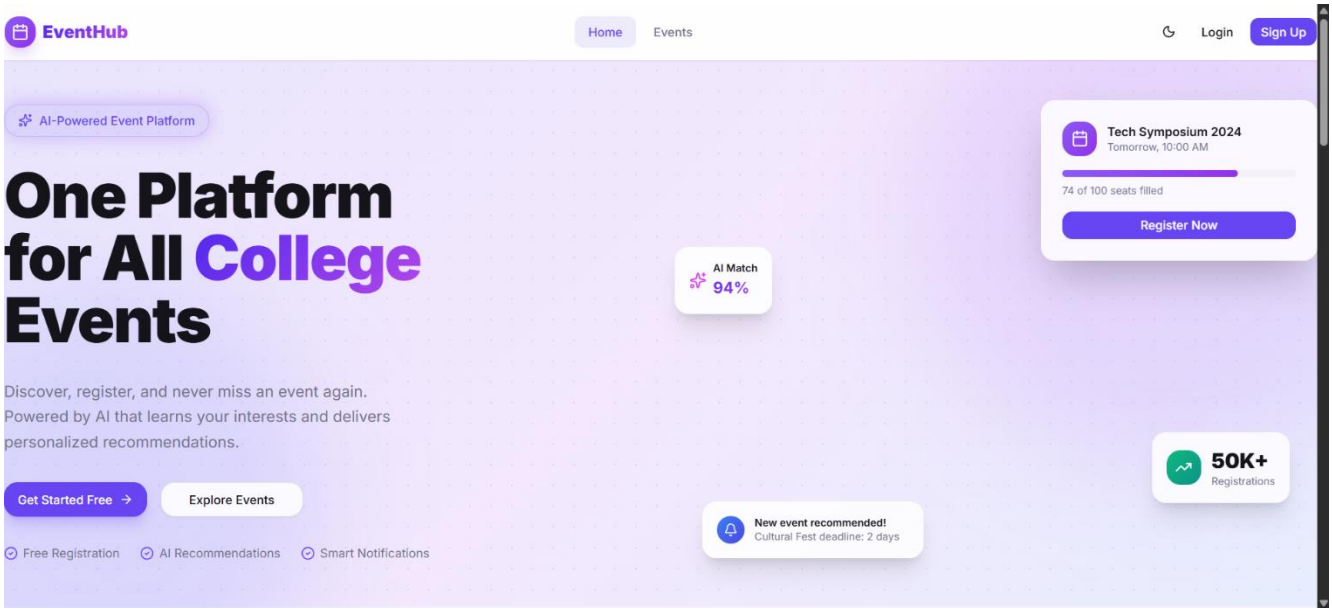
```
College Event Aggregator/
|
+-- app/                          # Frontend Application
|   +-- public/                   # Static assets
|   +-- src/
|   |   +-- components/           # Reusable UI components
|   |   |   +-- ui/               # shadcn/ui primitives
|   |   |   +-- layout/           # MainLayout, AuthLayout
|   |   |   +-- shared/           # Cards, Filters, etc.
|   |   +-- hooks/                # Custom React hooks
|   |   |   +-- useAuth.tsx       # Authentication context
|   |   |   +-- useTheme.tsx      # Theme management
|   |   +-- lib/                  # Utilities
|   |   |   +-- api.ts            # API client (301 lines)
|   |   |   +-- utils.ts          # Helper functions
|   |   +-- pages/                # Page components
|   |   |   +-- HomePage.tsx
|   |   |   +-- EventsPage.tsx
|   |   |   +-- EventDetailPage.tsx
|   |   |   +-- LoginPage.tsx
|   |   |   +-- RegisterPage.tsx
|   |   |   +-- DashboardPage.tsx
|   |   |   +-- CreateEventPage.tsx
|   |   |   +-- MyRegistrationsPage.tsx
|   |   |   +-- NotificationsPage.tsx
|   |   |   +-- ProfilePage.tsx
|   |   |   +-- AdminDashboardPage.tsx
|   |   |   +-- PendingApprovalsPage.tsx
|   |   |   +-- UserManagementPage.tsx
|   |   |   +-- NotFoundPage.tsx
|   |   +-- types/                # TypeScript interfaces
|   |   |   +-- index.ts          # 324 lines, 30+ interfaces
|   |   +-- App.tsx               # Root component with routing
|   |   +-- main.tsx              # Entry point
|   +-- package.json
|   +-- tsconfig.json
|   +-- vite.config.ts
|   +-- tailwind.config.ts
|
+-- backend/                      # Backend API Server
|   +-- prisma/
|   |   +-- schema.prisma         # Database schema definition
|   |   +-- migrations/           # SQL migration files
|   +-- src/
|   |   +-- controllers/          # Business logic
|   |   |   +-- auth.controller.ts      # 424 lines
```

```
|   |   |   +-- event.controller.ts     # 651 lines
|   |   |   +-- registration.controller.ts
|   |   |   +-- notification.controller.ts
|   |   |   +-- dashboard.controller.ts
|   |   |   +-- admin.controller.ts
|   |   |   +-- user.controller.ts
|   |   +-- middleware/           # Cross-cutting concerns
|   |   |   +-- auth.middleware.ts       # 277 lines
|   |   |   +-- error.middleware.ts
|   |   |   +-- notFound.middleware.ts
|   |   +-- routes/               # API route definitions
|   |   |   +-- auth.routes.ts
|   |   |   +-- event.routes.ts
|   |   |   +-- registration.routes.ts
|   |   |   +-- notification.routes.ts
|   |   |   +-- dashboard.routes.ts
|   |   |   +-- admin.routes.ts
|   |   |   +-- user.routes.ts
|   |   |   +-- ai.routes.ts
|   |   +-- services/             # Background services
|   |   |   +-- notification.service.ts
|   |   +-- utils/                # Utilities
|   |   |   +-- seed.ts           # Database seeder
|   |   |   +-- logger.ts         # Winston configuration
|   |   +-- server.ts             # Entry point (184 lines)
|   +-- package.json
|   +-- tsconfig.json
|   +-- .env
|
+-- ai-service/                   # AI Microservice
|   +-- src/
|   |   +-- main.py               # FastAPI app (449 lines)
|   |   +-- services/
|   |       +-- recommendation.py # Hybrid engine (361 lines)
|   |       +-- search.py         # NLP search (351 lines)
|   |       +-- categorization.py # Category predictor
|   |       +-- notification_scorer.py
|   +-- requirements.txt
|   +-- .env
|
+-- docs/                         # Documentation
|   +-- Final_Project_Report.md
|
+-- Web Images/                   # Screenshots
    +-- Homepage.png
    +-- Events.png
    +-- Login.png
```

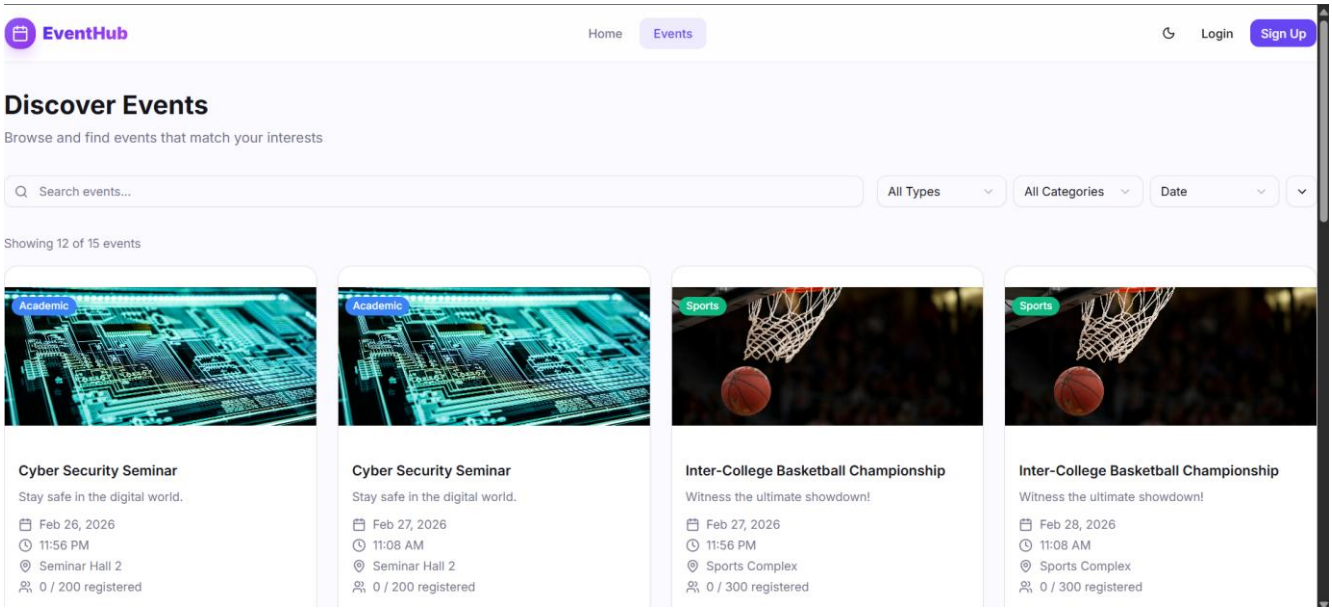
Appendix B: Screenshots

Screenshots of the running application are available in the `Web Images/` directory:

- **S.1: Home Page** – Displays the landing page with featured events, search bar, and category filters



- **S.2: Events Page** – Shows the full event directory with sorting, filtering, and pagination controls.



- **S.3: Login and Sign Up Forms** – Authentication modals with form validation and role selection.

